Lean Proof Strategy Proposal for the Scheduling Rules Document

Stuart Swan, Platform Architect, Siemens EDA

20 May 2026

0. Goal and scope

The goal is to formalize the scheduling rules document's **mathematical proof core** in Lean: trace compatibility, witness construction, cutpoint correspondence, elaboration, and one-way liveness/deadlock preservation. The Lean proof would certify the formalized theorem stack under the stated assumptions, not every informal prose statement in the document.

The scheduling rules document's own framing is conditional: the formal results depend on assumptions, observation-boundary choices, witness-selection conditions, and liveness/progress premises.

The Lean development should therefore be organized around explicit theorem instances and assumption bundles, rather than hidden global axioms.

Notation and signature status. The Lean-like declarations in this strategy are intended to fix proof architecture, theorem dependencies, and object domains. Unless a declaration is explicitly identified as a final Lean signature, it should be read as schematic pseudo-Lean. In particular, channel, process, side, prefix, and bucket types must be unified in the concrete development through the selected ComparisonInstance, the side-parametric SystemOfSide map, and the shared proof-internal alphabet C.Sigma. This note is not a relaxation of the proof obligations; it prevents schematic notation such as Channel, C.Sys_ref.Channel, Prefix, and InfiniteExecutionBucketed from being mistaken for already type-checking Lean code.

The primary scheduling rules document on which this doc is based is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

The latest version of this Lean proof strategy document is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/Lean_proof_strategy.pdf

The Matchlib examples kit that contains training slides, examples, etc., is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/tree/master

---

1. Central comparison object

The proof should be parameterized by a selected comparison instance:

```
structure ComparisonInstance where
Sys_ref : System
RTL_B : System
-- Shared proof-internal observable alphabet for the Sys_ref / RTL_B comparison.
-- Both sides emit events in this same Σ schema.
Sigma : Alphabet
-- Optional outer DUT/testbench-visible projection alphabet.
-- In ordinary cases this may be the identity projection from Sigma.
Sigma_DUT : Alphabet
sigmaDUTProjection : ProjectionMap Sigma Sigma_DUT
env : ReactiveEnvironment Sigma
refChoice : SysRefChoice
-- The explicit Sys_ref field and the reference-choice record must name the
-- same source reference system. This prevents theorem instances from using
-- C.Sys_ref in one definition and C.refChoice.Sys_ref in another while silently
-- referring to different systems.
sys_ref_consistent :
Sys_ref = refChoice.Sys_ref

-- Optional modular combinational model used by R2C.
-- The R2C semantics are still stated over a composed cycle-level
-- combinational fixed point, but that composed network is obtained from
-- explicit per-process combinational components. This preserves the document's
-- local/compositional proof style while still allowing a single global
-- ε-quiescent fixed-point valuation μ_t to be used for observable R2C events.
combModel? : Option CombComposition

assumptions : AssumptionBundle
```

witness : WitnessBundle
contracts : ModelingContracts

The comparison relation between Sys_ref and RTL_B is defined over the single shared proof-internal alphabet C.Sigma. The reference and post-HLS executions may have different internal ε-structure, bucket decomposition, or implementation state, but their observable events are encoded in the same Σ event schema before CompatibleP / CompatibleSys is applied.

Sigma_DUT is not a second post-side alphabet. It is only the chosen outer DUT/testbench-visible projection of C.Sigma. In elaborated cases, introduced proof-internal staged/bundle/join events may be hidden or accounted for by sigmaDUTProjection / Π_elab when projecting to Sigma_DUT, while the proof-internal compatibility theorem still ranges over C.Sigma.

Sys_ref is the chosen source reference model for the theorem instance:
- ordinary bounded-FIFO case: Sys_ref = Sys_B;
- elaborated case: Sys_ref = Sys_B^elab(E) for a well-formed elaboration package E.

The refChoice field records why this Sys_ref is the prescribed source reference model, and sys_ref_consistent enforces that C.Sys_ref and C.refChoice.Sys_ref are the same system. This follows the updated Appendix Q direction: the formal theorems do not require a unique algorithm that constructs Sys_B^elab; instead, the chosen Sys_ref is part of the theorem instance, but the theorem instance may not contain two inconsistent reference systems.

---

2. Foundation layer: bucketed executions, not flat traces

The primary execution model should be **cycle-bucketed**, not a flat sequence of eps | tick | sigma e. The reason is that the document allows multiple same-cycle Σ-events without imposing artificial order among independent same-cycle events. The earlier review correctly noted that a flat trace would either impose a fake linearization or require an added bucket abstraction anyway.

Use:

structure State where
  cycle : Nat

epsPos : Nat
store : Store

with explicit advancement rules:
-- Intra-cycle ε-normalization steps advance epsPos only.
-- Same-cycle Σ-events advance epsPos only.
-- Clock ticks advance cycle and reset epsPos.
-- Cycle-advancing silent progress is represented at the bucket level: a
-- CycleBucket with sigmaEvents = ∅ may still consume a real clock cycle and
-- may represent HLS-inserted FSM latency, arbitration/bookkeeping latency,
-- failed non-blocking polls, or pipeline advances that emit no Σ-event.

Define buckets:
structure CycleBucket (Σ : Type) where
cycle : Nat
startState : State
epsPrefix : List EpsStep
-- The Σ-events committed in this clock cycle.
-- This is an unordered finite set. A CycleBucket does not carry an intrinsic
-- sequence or bucket-local partial order among same-cycle events.
sigmaEvents : Finset Σ
epsSuffix : List EpsStep
endState : State
endQuiescent : Prop
def ExecutionBucketed (Σ : Type) :=
List (CycleBucket Σ)
def InfiniteExecutionBucketed (Σ : Type) :=
Nat → CycleBucket Σ

Bucket well-formedness predicates, not the Finset representation alone, enforce per-cycle scalar constraints such as at most one Push_c and at most one Pop_c per channel in a cycle. Required causal order is not stored in CycleBucket. It is derived globally from observable units and the HappensBefore relation generated by E1–E5, channel semantics, synchronization semantics, R2/R2C signal anchoring, atomic-unit constraints, witness-control dependencies, and elaboration-projection constraints.

This provides the right foundation for:
• ε-quiescent cutpoints;

• rendezvous same-cycle commits;
• R2C combinational fixed-point buckets;
• J.1 ideal induction over observable units and the global mandated happens-before relation;
• B2/B3 temporal reasoning over infinite bucketed executions.

Important separation: ExecutionBucketed is the semantic carrier for state advancement, ε-quiescence, occupancy updates, R2C fixed points, and fairness/progress reasoning. It is not itself the cross-model trace-compatibility predicate.

In particular, system-level compatibility τ_ref,system ≈_sys τ_post,system must not be defined as equality of CycleBucket lists, equality of bucket cycle numbers, or bucket-by-bucket equality of sigmaEvents. The document's Appendix J compatibility relation is an ordered cross-model
observable-compatibility predicate. It preserves the explicitly required happens-before / ≺_J constraints, atomic multi-endpoint units, E1–E5 obligations, and matched value labels, while allowing independent observable units to commute or to be grouped into different cycle buckets in Sys_ref and RTL_B. Same-cycle membership in a CycleBucket is therefore a timing fact for one execution, not a source of intrinsic order.

Thus:
• bucket equality may be required inside explicitly atomic units, such as rendezvous Push/Pop pairs and SyncChannel/ac_sync handshakes;
• bucket decomposition may be used to prove local state-transition and enablement facts;
• but CompatibleSys / ≈_sys is defined over observable units and their mandated partial order, not over identical bucket decompositions.

---

3. Events, operation instances, and source order

Define dynamic operation universes:

```
inductive MsgKind
| Push
| Pop
| PushNB
| PopNB
```

```
def MsgKind.isBlocking : MsgKind → Bool
| MsgKind.Push => true
| MsgKind.Pop => true
| MsgKind.PushNB => false
| MsgKind.PopNB => false

structure DynMsgInstance where
proc : Process
channel : Channel
kind : MsgKind -- Push, Pop, PushNB, PopNB
sourceLoc : SourceLoc
instanceId : Nat

def Blocking (q : DynMsgInstance) : Prop :=
q.kind.isBlocking = true

def NonBlocking (q : DynMsgInstance) : Prop :=
q.kind.isBlocking = false

structure FrontierItem where
  proc      : Process
  itemKind   : FrontierItemKind
  sourceLoc  : SourceLoc
  instanceId : Nat
```

Core source-order objects are witness-specific. They range only over dynamic operation instances that actually occur in the compared execution / witness. Untaken branches and unexecuted loop iterations are not members of this dynamic universe.

```
Qexec : Process → Set DynMsgInstance
QOmega : Process → Set DynMsgInstance
Omega : Process → Set FrontierItem
```

Qexec,P is the execution-level dynamic-instance universe. It includes executed failed non-blocking polls that may produce no Σ-event.

Omega,P is the frontier/order universe used by Front, NextFront, WFG, E3/E5, BoundaryReq attribution, and source-order reasoning. Failed NB-only executed

polls are not included in Omega,P merely because they executed.

QOmega,P denotes the executed dynamic message-operation-instance slice induced by Omega,P. Since Qexec,P is a set of DynMsgInstance and Omega,P is a set of FrontierItem, this is not a direct set intersection. It is defined through the message-instance projection from frontier items:

QOmega,P :=
{ q : DynMsgInstance |
q ∈ Qexec,P ∧
∃ x : FrontierItem,
x ∈ Omega,P ∧
MsgInstanceOfFrontierItem C P x = some q }

```
def MsgInstanceOfFrontierItem
(C : ComparisonInstance)
(P : Process)
(x : FrontierItem) : Option DynMsgInstance :=
...

-- R2/E2 signal anchors include both explicit synchronization calls and the
-- process-boundary synchronization events START_P and FINISH_P. This avoids
-- making signal reads before the first explicit wait(), or signal writes after
-- the last explicit wait(), ill-typed.
inductive SyncAnchor
| start (P : Process)
| call (s : SyncCall)
| finish (P : Process)

def SyncAnchorClock
(C : ComparisonInstance)
(W : WitnessBundle C)
(a : SyncAnchor) : Cycle :=
match a with
| SyncAnchor.start P => StartClock C W P
| SyncAnchor.call s => SyncClock C W s
| SyncAnchor.finish P => FinishClock C W P
```

```
-- A SyncAnchor may be used by pred_S / succ_S only when the corresponding
-- synchronization event is actually present on the dynamic path represented by
-- the witness universe. In particular, START_P is not an implicit free anchor:
-- it is usable only when the modeling convention includes START_P as a real
-- synchronization event in S. FINISH_P is usable only for an actually
-- terminating process whose FINISH_P event is realized.
def SyncAnchorRealizedInUniverse
(C : ComparisonInstance)
(P : Process)
(W : WitnessBundle C)
(Ω : Set FrontierItem)
(a : SyncAnchor) : Prop :=
match a with
| SyncAnchor.start P' =>
P' = P ∧
∃ x,
x ∈ Ω ∧
StartSyncFrontierItem C P x ∧
ExecutedInWitness C W P x
| SyncAnchor.call s =>
SyncCallOwnedByProcess C P s ∧
∃ x,
x ∈ Ω ∧
SyncCallFrontierItem C P s x ∧
ExecutedInWitness C W P x
| SyncAnchor.finish P' =>
P' = P ∧
ProcessTerminatesInWitness C W P ∧
∃ x,
x ∈ Ω ∧
FinishSyncFrontierItem C P x ∧
ExecutedInWitness C W P x

structure DynamicUniverse
(C : ComparisonInstance)
(P : Process)
(W : WitnessBundle C) where
Q_exec : Set DynMsgInstance
```

Ω : Set FrontierItem
Q_Ω : Set DynMsgInstance

qomega_characterization :
∀ q,
q ∈ Q_Ω ↔
q ∈ Q_exec ∧
∃ x,
x ∈ Ω ∧
MsgInstanceOfFrontierItem C P x = some q

message_frontier_items_have_exec_instance :
∀ x q,
x ∈ Ω →
MsgInstanceOfFrontierItem C P x = some q →
q ∈ Q_exec ∧ q ∈ Q_Ω

qomega_items_have_frontier_item :
∀ q,
q ∈ Q_Ω →
∃ x,
x ∈ Ω ∧
MsgInstanceOfFrontierItem C P x = some q

executed_only :
∀ x,
x ∈ Ω →
ExecutedInWitness C W P x

leP :
{x : FrontierItem // x ∈ Ω} →
{y : FrontierItem // y ∈ Ω} →
Prop

leP_partial_order :
PartialOrder leP

prefS :
SyncCall →
Set {x : FrontierItem // x ∈ Ω}
suffS :
SyncCall →
Set {x : FrontierItem // x ∈ Ω}

-- R2/E2 signal-anchor maps for sequential-process signal IO.
-- pred_S is defined for ordinary sequential-process signal Reads.
-- succ_S is defined for ordinary sequential-process signal Writes.
-- The anchor type is SyncAnchor, not SyncCall, because START_P may be the
-- closest preceding anchor for an initial Read and FINISH_P may be the closest
-- succeeding anchor for a terminal Write.
pred_S :
{x : FrontierItem // x ∈ Ω} →
Option SyncAnchor
succ_S :
{x : FrontierItem // x ∈ Ω} →
Option SyncAnchor

read_anchor_defined :
∀ x,
SequentialSignalReadItem x.val →
pred_S x ≠ none ∧
succ_S x = none
write_anchor_defined :
∀ x,
SequentialSignalWriteItem x.val →
succ_S x ≠ none ∧
pred_S x = none
pred_S_anchor_realized :
∀ x a,
SequentialSignalReadItem x.val →
pred_S x = some a →
SyncAnchorRealizedInUniverse C P W Ω a
succ_S_anchor_realized :
∀ x a,
SequentialSignalWriteItem x.val →

succ_S x = some a →
SyncAnchorRealizedInUniverse C P W Ω a
pred_S_is_closest_preceding_anchor :
∀ x a,
SequentialSignalReadItem x.val →
pred_S x = some a →
ClosestPrecedingDynamicSyncAnchor C P W a x.val
succ_S_is_closest_succeeding_anchor :
∀ x a,
SequentialSignalWriteItem x.val →
succ_S x = some a →
ClosestSucceedingDynamicSyncAnchor C P W a x.val

signal_read_ordered_at_pred_S :
∀ x a,
SequentialSignalReadItem x.val →
pred_S x = some a →
SignalItemClock C W x.val = SyncAnchorClock C W a

signal_write_ordered_at_succ_S :
∀ x a,
SequentialSignalWriteItem x.val →
succ_S x = some a →
SignalItemClock C W x.val = SyncAnchorClock C W a

MsgInstanceOfFrontierItem is defined only for frontier items that correspond to message-operation dynamic instances. Non-message frontier items, such as pure synchronization or signal-anchor items, map to none. Signal-anchor items instead use the explicit pred_S / succ_S fields above. For sequential processes, pred_S places each ordinary signal Read at its closest
preceding SyncAnchor, and succ_S places each ordinary signal Write at its closest succeeding SyncAnchor. A SyncAnchor may be START_P, an explicit synchronization call, or FINISH_P. These anchors, not raw lexical position, determine the signal item's formal position for ≤_P / <_P, Done_P, Remain_P, NextFront, E2SignalAnchorOrder, and CompatibleP / CompatibleSys reasoning.

The field Q_Ω is therefore not a second independent universe: it is exactly the set of executed dynamic message-operation instances q ∈ Q_exec for which some

message frontier item x ∈ Ω satisfies MsgInstanceOfFrontierItem C P x = some q. This keeps Q_exec, Ω, Q_Ω, and the signal-anchor universe type-distinct while making their relationships explicit for BoundaryReq, Front, NextFront, WFG, E2, E3/E5 issue-order reasoning, and R2/R2C signal visibility.

Thus source order ≤_P is not a static total order over syntactic program statements. It is a partial order over the witness-bundle-specific dynamic frontier-item universe Ω. The selector component W.selector determines ε-modeled source choices, while W.nbMap records the dynamic-instance correspondence for failed non-blocking polls that produce no Σ-event.

Non-blocking calls need explicit modeling:
· failed PushNB / PopNB polls are executed dynamic instances in Qexec, but produce no Σ-event;
· successful non-blocking calls produce the corresponding committed Push_c(v) / Pop_c(v) event;
· repeated NB calls are distinct dynamic source instances even if a request signal remains physically asserted.
This matches the document's issue/commit treatment of NB calls.

structure NonBlockingInstanceWF
(C : ComparisonInstance)
(P : Process)
(W : WitnessBundle C)
(U : DynamicUniverse C P W) where

nb_instances_are_nonpersistent :
∀ q,
q ∈ U.Q_exec →
NonBlocking q →
¬ PersistentBlockingRequestInstance C P q

nb_reexecution_distinct_instances :
∀ q_1 q_2 t_1 t_2,
q_1 ∈ U.Q_exec →
q_2 ∈ U.Q_exec →
NonBlocking q_1 →
NonBlocking q_2 →

SameSourceNBPollSite q_1 q_2 →
ExecutedAtCycle C W q_1 t_1 →
ExecutedAtCycle C W q_2 t_2 →
$t_1 \neq t_2$ →
$q_1 \neq q_2$

nb_physical_assertion_does_not_identify_instances :
∀ q_1 q_2 t_1 t_2,
$q_1 \in$ U.Q_exec →
$q_2 \in$ U.Q_exec →
NonBlocking q_1 →
NonBlocking q_2 →
SameEndpoint q_1 q_2 →
EndpointRequestContinuouslyAsserted C W q_1.channel t_1 t_2 →
ExecutedAtCycle C W q_1 t_1 →
ExecutedAtCycle C W q_2 t_2 →
$t_1 \neq t_2$ →
$q_1 \neq q_2$

nb_not_fair_action :
∀ q,
q ∈ U.Q_exec →
NonBlocking q →
¬ ∃ h, FairAction.push q h ∨ FairAction.pop q h

The last field is only a type-level sanity condition for the fairness layer:
B2 fairness ranges over persistent blocking Push/Pop requests and real synchronization calls. A successful NB call may still contribute the ordinary committed Push_c / Pop_c Σ-event, but the NB call instance itself does not become a persistent FairAction.

---

4. Reactive environment and modeling contracts

4.1 Reactive environment

“Fixed stimulus” should be modeled as a causal global strategy over Σ-history, not merely as a fixed stream of per-process inputs.

structure ReactiveEnvironment (Σ : Type) where
 inputAt : SigmaHistory Σ → ExternalInputs
 causal :
  ∀ h_1 h_2,
   SamePrefix h_1 h_2 →
   inputAt h_1 = inputAt h_2

This directly addresses the earlier issue that reactive stimulus must be interpreted as the same global Σ-causal behavior, not a function of local process history.

4.2 Modeling contracts

Direct-input contracts and array-access contracts should be parallel top-level modeling contracts, not partly folded into RRules and partly left separate.

structure ModelingContracts (C : ComparisonInstance) where
 directInputs : DirectInputContract C
 arrays : ArrayAccessMappingRules C

This keeps RRules close to the document's actual R-rule set and treats direct-input and external-array obligations as environment/modeling contracts.

---

5. Direct-input contract and late sampling

The direct-input contract should contain only primitive environmental stability/update obligations. The fact that late sampling preserves the logical anchor should be a **derived theorem**, not a separate hypothesis.

The document says hls_direct_input permits late physical sampling because the environment holds the signal stable after reset, while hls_direct_input_sync permits controlled updates only at the designated synchronization handshake. The document also states that logical signal Read/Write events are timestamped at the E2 anchor, not at any later physical RTL sampling cycle; late sampling is ε-internal.

Define:

The direct-input stability rule applies only within a single reset epoch. The scheduling document explicitly permits dynamic resetting during normal HW operation, including resetting of direct-input-controlled regions.

def NoRelevantResetBetween
(C : ComparisonInstance)

```
(sig : Signal)
(t₁ t₂ : Cycle) : Prop :=
¬ ∃ r,
t₁ < r ∧
r ≤ t₂ ∧
ReceiverResetAffectsSignal C sig r

structure DirectInputContract (C : ComparisonInstance) where
stableWithinResetEpoch :
∀ sig t₁ t₂,
DirectInput sig →
AfterAllReceiversOutOfReset C sig t₁ →
t₁ ≤ t₂ →
NoRelevantResetBetween C sig t₁ t₂ →
InputValue C.env sig t₁ = InputValue C.env sig t₂
syncControlledUpdateOnlyAtSync :
∀ sig s t,
DirectInputSyncControlled sig s →
ValueChangesAt C.env sig t →
SyncHandshakeOccursAt C s t
```

Then prove:

```
theorem lateSamplingPreservesLogicalAnchor
(C : ComparisonInstance)
(hDI : DirectInputContract C)
(sig : Signal)
(anchor sampleT : Cycle)
(hIsDI : DirectInput sig)
(hOutOfReset :
AfterAllReceiversOutOfReset C sig anchor)
(hWindow : DirectInputAnchorWindow C sig anchor sampleT)
(hLe : anchor ≤ sampleT)
(hNoReset :
NoRelevantResetBetween C sig anchor sampleT) :
InputValue C.env sig anchor =
InputValue C.env sig sampleT :=
hDI.stableWithinResetEpoch
sig
anchor
sampleT
```

hIsDI
hOutOfReset
hLe
hNoReset

Bucket-location rule:
For sequential processes, the logical direct-input Read(sig,val) / Write(sig,val) event belongs only to the E2 anchor bucket β_anchor. A physical late sample, if modeled, is an ε-step in a later bucket. The late ε-step creates no new Σ-event. lateSamplingPreservesLogicalAnchor is used only to justify the value recorded in the logical sequential anchor bucket.

This sequential anchoring rule is not applied to combinational-process signal IO. Combinational processes do not have pred_S / succ_S synchronization anchors. Their observable direct-input Read/Write labels, when any direct input participates in combinational signal IO, are recorded in the R2C bucket corresponding to the cycle's ε-quiescent combinational fixed point. Intermediate physical samples, delta-cycle reads/writes, redundant re-evaluations, and transient values before that fixed point are ε-steps and are not separately Σ-visible.

structure DirectInputBucketWF (C : ComparisonInstance) where
sequential_read_recorded_only_at_anchor :
∀ sig val anchor sampleT β,
DirectInput sig →
SequentialSignalReadEvent (LogicalReadEvent sig val) →
LogicalReadEvent sig val ∈ BucketSigmaEvents C β →
DirectInputAnchorWindow C sig anchor sampleT →
β = AnchorBucket C anchor

The late-sample value agreement field is reset-epoch local. It is derived from stableWithinResetEpoch and may be used only when no relevant dynamic reset occurs between the logical anchor cycle and the physical sample cycle.

sequential_write_recorded_only_at_anchor :
∀ sig val anchor β,
DirectInput sig →
SequentialSignalWriteEvent (LogicalWriteEvent sig val) →
LogicalWriteEvent sig val ∈ BucketSigmaEvents C β →

β = AnchorBucket C anchor

late_sample_is_epsilon_only :
∀ sig anchor sampleT,
DirectInputAnchorWindow C sig anchor sampleT →
PhysicalLateSample C sig sampleT →
EpsilonStepOnly C sig sampleT ∧
¬ ∃ val β,
PhysicalSampleEvent sig val sampleT ∈ BucketSigmaEvents C β

late_sample_value_matches_anchor :
∀ sig anchor sampleT,
DirectInput sig →
AfterAllReceiversOutOfReset C sig anchor →
DirectInputAnchorWindow C sig anchor sampleT →
anchor ≤ sampleT →
NoRelevantResetBetween C sig anchor sampleT →
InputValue C.env sig anchor = InputValue C.env sig sampleT

This explicitly reconciles the bucket model with direct-input late sampling: CompatibleP and CompatibleSys compare the logical Σ-visible direct-input Read/Write events at the appropriate logical bucket — the E2 anchor bucket for sequential-process signal IO, or the R2C ε-quiescent fixed-point bucket for combinational-process signal IO — never the later physical sample micro-step.

Combinational-process signal IO uses R2C, not the direct-input late-sampling contract. The R2C bucket-placement rule is therefore a general combinational-process well-formedness obligation. It applies to every combinational-process signal Read/Write event in the compared Σ trace, regardless of whether the signal is marked DirectInput.

structure CombinationalBucketWF (C : ComparisonInstance) where
combinational_read_recorded_only_at_fixed_point :
∀ sig val β,
CombReadWriteEvent (LogicalReadEvent sig val) →
LogicalReadEvent sig val ∈ BucketSigmaEvents C β →
R2CFixedPointBucket C β ∧
EventReadsWritesBucketFixedPoint C (LogicalReadEvent sig val) β

combinational_write_recorded_only_at_fixed_point :
∀ sig val β,
CombReadWriteEvent (LogicalWriteEvent sig val) →
LogicalWriteEvent sig val ∈ BucketSigmaEvents C β →
R2CFixedPointBucket C β ∧
EventReadsWritesBucketFixedPoint C (LogicalWriteEvent sig val) β

---

6. Temporal logic layer for B2/B3/B4/B5/B6

B2 and B3 should not be opaque Prop fields. They are temporal assumptions over infinite executions.

The document defines B2 as weak fairness: if an action remains continuously enabled from some cycle onward, the scheduler must eventually select it. It applies to Push, Pop, and Sync operations. It also defines B3 as the P_push(c) / P_pop(c) channel-progress obligations.

Define temporal operators:

```
def AlwaysFrom
 (τ : InfiniteExecutionBucketed Σ)
 (t : Nat)
 (P : Nat → Prop) : Prop :=
 ∀ n, t ≤ n → P n


def EventuallyFrom
 (τ : InfiniteExecutionBucketed Σ)
 (t : Nat)
 (P : Nat → Prop) : Prop :=
 ∃ n, t ≤ n ∧ P n
```

Concrete fairness actions:

```
-- B2 fairness ranges only over scheduler-selectable operations.
-- For message operations, this means persistent blocking Push and Pop boundary
-- requests. Non-blocking PushNB/PopNB polls are not FairAction cases: failed NB
-- polls are ε-steps, and successful NB polls contribute the ordinary committed
-- Push/Pop Σ-event without creating a persistent scheduler-fairness obligation.
-- For synchronization, fairness ranges only over real synchronization calls
```

```
-- such as wait(), ac_wait, ac_sync, or SyncChannel calls. It does not range over
-- the auxiliary process-boundary anchors START_P and FINISH_P.
inductive FairAction
| push (q : DynMsgInstance) (h : q.kind = MsgKind.Push)
| pop (q : DynMsgInstance) (h : q.kind = MsgKind.Pop)
| sync (s : SyncCall)
```

Weak fairness:
Weak fairness is a side-parametric temporal assumption over the FairAction type above. For message operations, FairAction contains only blocking Push and blocking Pop dynamic instances. For synchronization operations, FairAction contains only real SyncCall instances, not arbitrary SyncAnchor values. Thus START_P and FINISH_P may still be used as formal R2/E2 signal anchors when they are realized in the dynamic universe, but they are not scheduler-fairness actions. Non-blocking PushNB/PopNB instances are excluded at the type level; they are handled through the NB execution/witness machinery and through ordinary committed Push/Pop Σ-events when successful.

Weak fairness applies to any valid infinite execution of the selected comparison side. In particular, it is not a post-only premise: the reference-side execution used by Exec_ref and the post-side execution used by RTL_B both carry the same WF/B2 obligation whenever that side is being used in a liveness/progress theorem.

```
def WeakFairness
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma) : Prop :=
∀ a t,
AlwaysFrom τ t (fun n => FairActionEnabled C side τ a n) →
EventuallyFrom τ t (fun n => FairActionSelected C side τ a n)
```

B3 channel-progress predicates should be defined in terms of continuous endpoint-level ready/valid requests, with operation attribution supplied separately. They should not require that the complementary endpoint's continuous request be witnessed by one persistent dynamic message-operation instance.

A blocking request is persistent from cycle t through its commit cycle inclusive

when the same dynamic operation instance q keeps its BoundaryReq asserted throughout that interval. For Push operations, persistence also includes payload stability while the request is outstanding, including the commit cycle itself:

```
def PersistentRequestFrom
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(q : DynMsgInstance)
(t : Nat) : Prop :=
Blocking q ∧
∀ n,
t ≤ n →
¬ CommitsBefore C side τ q n →
BoundaryReqAt C side τ q n ∧
(q.kind = MsgKind.Push →
PayloadAt C side τ q n = PayloadAt C side τ q t)
```

PersistentRequestFrom remains the right predicate for a single outstanding blocking Push or Pop. It is deliberately not used to collapse a sequence of executed non-blocking polls into one dynamic instance. Repeated PushNB/PopNB polls are distinct source-level dynamic instances, even if the concrete endpoint request signal remains asserted across adjacent cycles.

```
inductive EndpointDirection
| push
| pop

def DirectionOf
(q : DynMsgInstance) : EndpointDirection :=
match q.kind with
| MsgKind.Push   => EndpointDirection.push
| MsgKind.PushNB => EndpointDirection.push
| MsgKind.Pop    => EndpointDirection.pop
| MsgKind.PopNB  => EndpointDirection.pop

def IsPushKind
(k : MsgKind) : Prop :=
k = MsgKind.Push ∨ k = MsgKind.PushNB

def IsPopKind
(k : MsgKind) : Prop :=
k = MsgKind.Pop ∨ k = MsgKind.PopNB
```

```
def EndpointRequestAssertedAt
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(dir : EndpointDirection)
(n : Nat) : Prop :=
match dir with
| EndpointDirection.push => EndpointValidAssertedAt C side τ c n
| EndpointDirection.pop  => EndpointReadyAssertedAt C side τ c n


structure EndpointRequestWitness
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(dir : EndpointDirection)
(n : Nat) where
endpoint_asserted :
EndpointRequestAssertedAt C side τ c dir n
source_attributed :
∃ q,
q.channel = c ∧
DirectionOf q = dir ∧
BoundaryReqAt C side τ q n
payload_present_if_push :
dir = EndpointDirection.push →
∃ v, EndpointPayloadAt C side τ c n = some v

-- Endpoint request continuity is not a forever-after-t predicate. It is scoped
-- to the interval before the relevant discharge event D. After D occurs, the
-- endpoint may drop the request or change payload for the next operation.
def EndpointRequestHoldsUntilDischarged
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(dir : EndpointDirection)
(t : Nat)
(D : Nat → Prop) : Prop :=
HoldsUntilDischarged τ t
(fun n =>
EndpointRequestWitness C side τ c dir n)
```

```
D

-- Push payload stability is also scoped only to the outstanding interval. A
-- successful transfer may be followed by a deasserted request or a different
-- payload for a later operation.
def PushPayloadStableUntilDischarged
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(t : Nat)
(D : Nat → Prop) : Prop :=
∃ v,
HoldsUntilDischarged τ t
(fun n =>
EndpointPayloadAt C side τ c n = some v)
D

def ContinuousPushRequestUntilDischarged
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(t : Nat)
(D : Nat → Prop) : Prop :=
EndpointRequestHoldsUntilDischarged C side τ c EndpointDirection.push t D ∧
PushPayloadStableUntilDischarged C side τ c t D

def ContinuousPopRequestUntilDischarged
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(t : Nat)
(D : Nat → Prop) : Prop :=
EndpointRequestHoldsUntilDischarged C side τ c EndpointDirection.pop t D

def BufferedChannel
(C : ComparisonInstance)
(c : Channel) : Prop :=
0 < capacity C c

def RendezvousChannel
(C : ComparisonInstance)
```

```
(c : Channel) : Prop :=
capacity C c = 0

def SomePopCommitOn
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(n : Nat) : Prop :=
∃ qPop,
qPop.channel = c ∧
IsPopKind qPop.kind ∧
CommitAtCycle C side τ qPop n

def SomePushCommitOn
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(n : Nat) : Prop :=
∃ qPush,
qPush.channel = c ∧
IsPushKind qPush.kind ∧
CommitAtCycle C side τ qPush n

def MatchingTransferCommitOnChannel
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(n : Nat) : Prop :=
∃ qPush qPop,
qPush.channel = c ∧
qPop.channel = c ∧
IsPushKind qPush.kind ∧
IsPopKind qPop.kind ∧
MatchingTransferCommitOn C side τ qPush qPop n

def BufferedFullBothEndpointsRequesting
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(n : Nat) : Prop :=
```

```
BufferedChannel C c ∧
occAt c n = capacity C c ∧
EndpointRequestAssertedAt C side τ c EndpointDirection.push n ∧
EndpointRequestAssertedAt C side τ c EndpointDirection.pop n

def BufferedEmptyBothEndpointsRequesting
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(n : Nat) : Prop :=
BufferedChannel C c ∧
occAt c n = 0 ∧
EndpointRequestAssertedAt C side τ c EndpointDirection.pop n ∧
EndpointRequestAssertedAt C side τ c EndpointDirection.push n

def RendezvousBothEndpointsRequesting
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(n : Nat) : Prop :=
RendezvousChannel C c ∧
EndpointRequestAssertedAt C side τ c EndpointDirection.push n ∧
EndpointRequestAssertedAt C side τ c EndpointDirection.pop n

-- P holds continuously from t until the first discharge cycle.
-- This avoids the vacuity that would result from requiring the stalled condition
-- to remain true forever after the discharge has already occurred.
def HoldsUntilDischarged
(τ : InfiniteExecutionBucketed Σ)
(t : Nat)
(P : Nat → Prop)
(D : Nat → Prop) : Prop :=
∀ n, t ≤ n → (∀ k, t ≤ k → k < n → ¬ D k) → P n

def EventuallyDischargedFrom
(τ : InfiniteExecutionBucketed Σ)
(t : Nat)
(D : Nat → Prop) : Prop :=
∃ n, t ≤ n ∧ D n
```

Then:

```
def P_push
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel) : Prop :=
(BufferedChannel C c →
∀ t,
BufferedFullBothEndpointsRequesting C side τ c t →
ContinuousPushRequestUntilDischarged C side τ c t
(fun n =>
SomePopCommitOn C side τ c n) →
ContinuousPopRequestUntilDischarged C side τ c t
(fun n =>
SomePopCommitOn C side τ c n) →
EventuallyDischargedFrom τ t
(fun n =>
SomePopCommitOn C side τ c n))
∧
(RendezvousChannel C c →
∀ t,
RendezvousBothEndpointsRequesting C side τ c t →
ContinuousPushRequestUntilDischarged C side τ c t
(fun n =>
MatchingTransferCommitOnChannel C side τ c n) →
ContinuousPopRequestUntilDischarged C side τ c t
(fun n =>
MatchingTransferCommitOnChannel C side τ c n) →
EventuallyDischargedFrom τ t
(fun n =>
MatchingTransferCommitOnChannel C side τ c n))

def P_pop
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel) : Prop :=
(BufferedChannel C c →
∀ t,
BufferedEmptyBothEndpointsRequesting C side τ c t →
ContinuousPopRequestUntilDischarged C side τ c t
(fun n =>
SomePushCommitOn C side τ c n) →
ContinuousPushRequestUntilDischarged C side τ c t
(fun n =>
```

SomePushCommitOn C side τ c n) →
EventuallyDischargedFrom τ t
(fun n =>
SomePushCommitOn C side τ c n))
∧
(RendezvousChannel C c →
∀ t,
RendezvousBothEndpointsRequesting C side τ c t →
ContinuousPopRequestUntilDischarged C side τ c t
(fun n =>
MatchingTransferCommitOnChannel C side τ c n) →
ContinuousPushRequestUntilDischarged C side τ c t
(fun n =>
MatchingTransferCommitOnChannel C side τ c n) →
EventuallyDischargedFrom τ t
(fun n =>
MatchingTransferCommitOnChannel C side τ c n))

The removed HoldsUntilDischarged premises are not independent B3 assumptions. For bounded FIFOs, the needed full/empty persistence facts are derived from E4: if occAt c t = capacity C c and no Pop_c commits before n, then the channel remains full at n; dually, if occAt c t = 0 and no Push_c commits before n, then the channel remains empty at n. For rendezvous channels, the corresponding "both endpoints requesting until transfer" fact is exactly the conjunction of the two continuous endpoint-request premises together with RendezvousChannel C c.

Thus P_push and P_pop state only the real B3 progress obligation: once the appropriate blocked condition holds at t and both endpoint requests continue until the matching discharge event, that discharge event must eventually occur. The occupancy-stability facts needed inside proofs should be supplied as E4 derived lemmas rather than duplicated as extra hypotheses in B3.

This matches the document's conditional B3 reading: B3 forces progress only when both endpoints continuously request the matching transfer until the corresponding discharge event. A producer stalled on a full channel is not automatically a B3 violation if the consumer is not yet requesting the matching pop, and a consumer stalled on an empty channel is not automatically a B3 violation if the producer is not yet requesting the matching push.

The continuous-request premise is endpoint-level and interval-scoped. For blocking Push/Pop it may be witnessed by one persistent dynamic operation instance using PersistentRequestFrom through the commit cycle. For non-blocking polling loops it may instead be witnessed by a sequence of distinct executed PushNB/PopNB instances on the same endpoint, provided the endpoint request

remains continuously asserted until discharge and each cycle's assertion has BoundaryReq attribution to the appropriate executed dynamic instance.

The payload-stability premise for Push is likewise scoped only until discharge. After the discharge event, the producer may deassert valid or present a different payload for a later operation. Thus B3 tracks the ready/valid progress premise without requiring an endpoint request or payload to remain stable forever and without misidentifying multiple NB polls as one persistent q.

The scheduling document assumes B1 determinism only of the post-HLS RTL_B side. The source-side models Sys / Sys_B / Sys_ref may admit multiple ε- or latency-different executions having the same Σ-projection; witness selection and snooping resolve the required source-side correspondence when needed.

For a fixed comparison instance — including the fixed initial state and fixed reactive environment / external stimulus — B1 requires uniqueness of the labeled post-side Σ-trace, not uniqueness of the full internal ε-execution. Internal ε-steps, ε-depths, and other non-observable scheduling details may differ between valid post-side executions.

def PostDeterministicSigmaTrace
(C : ComparisonInstance) : Prop :=
$\forall\ \tau_1\ \tau_2$,
InfinitePostExecution C τ_1 →
InfinitePostExecution C τ_2 →
SigmaTraceOf C .post τ_1 =
SigmaTraceOf C .post τ_2

Constructive ε-normalization and finite-stutter obligations for B4/B5.
B4 and B5 should not be merely opaque Prop fields when used to mechanize Lemma I.1-style arguments. They are assumptions, but they must expose enough structure to prove that internal ε-evolution cannot hide an infinite internal tail before the next Σ-action or stable wait.

This Lean strategy distinguishes two implementation-level measurements that are both silent with respect to Σ:
• intra-cycle ε-normalization: ε-microsteps inside one CycleBucket, used to reach an ε-quiescent cutpoint within a cycle; and
• cycle-advancing silent progress: one or more real clock cycles whose buckets

contain no Σ-event for the process, used to model HLS-inserted FSM latency, arbitration/bookkeeping latency, failed non-blocking polls, and pipeline advances.

This distinction is only a Lean-internal decomposition. It is not a redefinition of B4. The main-document B4 premise remains a single finite ε-stutter bound for each process P: along any consecutive ε-only evolution in which P is internally advancing and is not externally stalled, P must reach either a Σ-action or a stable waiting state within a finite bound D_P.

Accordingly, ProcessEpsilonBound and IntraCycleNormalizationBound may remain separate implementation parameters, but FiniteEpsilonDepth must expose a derived unified B4 consequence. In a concrete Lean development, D_P may be chosen large enough to dominate both the cycle-advancing silent-progress bound and the intra-cycle normalization bound. The two internal bounds are useful for proof engineering, but together they must imply the single main-document B4 finite-stutter obligation.

```
def ProcessEpsilonBound
(C : ComparisonInstance)
(P : Process) : Nat :=
pipe_depth C P

def IntraCycleNormalizationBound
(C : ComparisonInstance)
(P : Process) : Nat :=
eps_norm_depth C P

-- Main-document B4 bound.
-- This is the single D_P-style finite ε-stutter bound exposed to the theorem
-- stack. The exact arithmetic expression is not important; the requirement is
-- that MainB4Bound dominate the Lean-internal cycle-progress and bucket
-- normalization bounds strongly enough to prove the unified B4 consequence.
def MainB4Bound
(C : ComparisonInstance)
(P : Process) : Nat :=
(ProcessEpsilonBound C P + 1) *
(2 * IntraCycleNormalizationBound C P + 1)
```

```
def IntraCycleEpsilonStep
(C : ComparisonInstance)
(side : Side)
(σ σ' : State) : Prop :=
InternalStep C side σ σ' ∧
SigmaEventsBetween C side σ σ' = ∅ ∧
σ'.cycle = σ.cycle

def IntraCycleEpsilonSegment
(C : ComparisonInstance)
(side : Side)
(σ₀ σ₁ : State)
(steps : List EpsStep) : Prop :=
EpsilonStepsFromTo C side σ₀ steps σ₁ ∧
∀ σ σ',
AdjacentInEpsilonSteps C side σ₀ steps σ₁ σ σ' →
IntraCycleEpsilonStep C side σ σ'


def StableWaitOrSigmaEnabled
(C : ComparisonInstance)
(side : Side)
(σ : State) : Prop :=
StableWaitCutpoint C side σ ∨
∃ e, SigmaEnabledAt C side e σ

-- Event ownership is side-parametric. In simple cases the ownership predicate
-- may reduce to a side-agnostic schema-level owner map over C.Sigma, but the
-- side parameter is still carried here so bucket membership, execution
-- well-formedness, and process attribution are checked in the same side context.
def SigmaCommitsForProcessInBucket
(C : ComparisonInstance)
(side : Side)
(P : Process)
(β : CycleBucket C.Sigma) : Prop :=
∃ e,
e ∈ β.sigmaEvents ∧
```

EventOwnedByProcess C side P e

def SigmaCommitsForProcessAt
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : InfiniteExecutionBucketed C.Sigma)
(n : Cycle) : Prop :=
SigmaCommitsForProcessInBucket C side P (τ n)

def ProcessSilentCycle
(C : ComparisonInstance)
(side : Side)
(P : Process)
(β : CycleBucket C.Sigma) : Prop :=
WellFormedCycleBucket C side β ∧
ProcessActiveInBucket C side P β ∧
¬ SigmaCommitsForProcessInBucket C side P β ∧
InternalProgressForProcess C side P β

def SilentCycleSegment
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : InfiniteExecutionBucketed C.Sigma)
(t_0 t_1 : Cycle) : Prop :=
$t_0 \le t_1$ ∧
∀ n,
$t_0 \le n$ →
$n < t_1$ →
ProcessSilentCycle C side P (τ n)

def StableWaitOrSigmaEnabledAtCycle
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : InfiniteExecutionBucketed C.Sigma)
(n : Cycle) : Prop :=

StableWaitOrSigmaEnabled C side ((τ n).endState) ∨
∃ e,
EventOwnedByProcess C side P e ∧
SigmaEnabledAt C side e ((τ n).endState)

structure FiniteEpsilonDepth (C : ComparisonInstance) where
main_B4_bound_dominates_cycle_bound :
∀ P,
ProcessEpsilonBound C P ≤ MainB4Bound C P

main_B4_bound_dominates_normalization_bound :
∀ P,
IntraCycleNormalizationBound C P ≤ MainB4Bound C P

-- Cycle-level consequence used by Lemma I.1-style arguments.
silent_cycle_bound :
∀ side τ P t_0 t_1,
InfiniteExecutionOfSide C side τ →
SilentCycleSegment C side P τ t_0 t_1 →
(∀ n,
t_0 ≤ n →
n < t_1 →
¬ StableWaitOrSigmaEnabledAtCycle C side P τ n) →
t_1 - t_0 ≤ ProcessEpsilonBound C P

-- Main-document B4 consequence: all consecutive ε-only evolution, including
-- any intra-cycle normalization ε-steps represented inside CycleBucket, is
-- bounded by a single D_P-style bound before the process either commits a
-- Σ-action or reaches stable wait.
unified_epsilon_stutter_bound :
∀ side τ P s,
InfiniteExecutionOfSide C side τ →
ConsecutiveEpsilonOnlySegment C side P τ s →
ProcessInternallyAdvancingThroughout C side P τ s →
NoExternalStallThroughout C side P τ s →
NoSigmaCommitOrStableWaitBeforeEnd C side P τ s →
EpsilonStepCount C side P τ s ≤ MainB4Bound C P

normalizes_or_reaches_sigma_or_wait :
∀ side τ P t_0,
InfiniteExecutionOfSide C side τ →
ProcessActiveInBucket C side P (τ t_0) →
BeginsInternalProgressTowardNextObservable C side P τ t_0 →
∃ n,
t_0 ≤ n ∧
n ≤ t_0 + MainB4Bound C P ∧
(SigmaCommitsForProcessAt C side P τ n ∨
StableWaitOrSigmaEnabledAtCycle C side P τ n)

-- Counts only ε-steps attributed to process P. A global CycleBucket may contain
-- ε-steps from several processes, so the per-process normalization bound must
-- be stated over the P-owned slice of the ε-prefix / ε-suffix, not over the
-- entire global list.
def ProcessEpsilonStepsIn
(C : ComparisonInstance)
(side : Side)
(P : Process)
(steps : List EpsStep) : Nat :=
CountEpsStepsOwnedByProcess C side P steps

-- Equality of the non-clock state components that may not change across an
-- idle quiescent bucket. A bucket may still advance the clock, so B5-style
-- idle/quiescent reasoning must not assert β.endState = β.startState unless
-- endState is defined before the tick. However, when the bucket has no
-- Σ-events and no ε-prefix/ε-suffix steps, the intra-cycle ε position must not
-- drift silently.
def NonClockStateEq
(C : ComparisonInstance)
($σ_0$ $σ_1$: State) : Prop :=
$σ_0$.store = $σ_1$.store ∧
$σ_0$.epsPos = $σ_1$.epsPos

structure QuiescentNormalization (C : ComparisonInstance) where
bucket_prefix_bounded :
∀ side β P,

```
WellFormedCycleBucket C side β →
ProcessEpsilonStepsIn C side P β.epsPrefix ≤ IntraCycleNormalizationBound C P

bucket_suffix_bounded :
∀ side β P,
WellFormedCycleBucket C side β →
ProcessEpsilonStepsIn C side P β.epsSuffix ≤ IntraCycleNormalizationBound C P

end_quiescent_sound :
∀ side β,
WellFormedCycleBucket C side β →
β.endQuiescent →
QuiescentAt C side β.endState

system_quiescence_bound :
Nat
system_quiescence_bound_is_max :
∀ P,
MainB4Bound C P ≤ system_quiescence_bound

-- Starting a bucket at an ε-quiescent state only says that the incoming state
-- has no pending internal normalization. It does not forbid Σ-events in the
-- bucket, and it does not forbid ε-suffix normalization caused by those Σ-events.
-- The no-further-ε conclusion is valid only for an idle bucket in which no
-- observable action is enabled and no external/environmental change wakes the
-- system.
idle_quiescent_bucket_has_no_epsilon_steps :
∀ side β,
WellFormedCycleBucket C side β →
QuiescentAt C side β.startState →
β.sigmaEvents = ∅ →
NoObservableActionEnabledAtState C side β.startState →
NoExternalChangeInBucket C side β →
β.epsPrefix = [] ∧
β.epsSuffix = [] ∧
NonClockStateEq C β.startState β.endState

bounded_system_quiescence :
```

∀ side τ t,
InfiniteExecutionOfSide C side τ →
NoObservableActionEnabledAtState C side ((τ t).startState) →
NoExternalChangeFrom C side τ t →
∃ n,
t ≤ n ∧
n ≤ t + system_quiescence_bound ∧
QuiescentAt C side ((τ n).endState) ∧
∀ m,
n ≤ m →
QuiescentAt C side ((τ m).startState) ∧
QuiescentAt C side ((τ m).endState) ∧
((NoExternalChangeFrom C side τ m ∧
NoObservableActionEnabledAtState C side ((τ m).startState)) →
(τ m).sigmaEvents = ∅ ∧
(τ m).epsPrefix = [] ∧
(τ m).epsSuffix = [])

The bounded_system_quiescence field is the Lean-level form of B5. It is triggered from a single cycle whose start state has no enabled observable action. If no external/environmental change wakes the system, the system must reach the ε-fixed point within the explicit bound max_P D_P, represented here by system_quiescence_bound.

After that point, later buckets remain idle only while the no-external-change condition continues to hold and no observable action becomes enabled again. Thus B5 is a bounded quiescence-closure premise from an idle start state, not a requirement to prove in advance that no observable action is disabled forever from the original cycle.

The field idle_quiescent_bucket_has_no_epsilon_steps is deliberately weaker than the invalid statement “QuiescentAt β.startState implies β.epsPrefix = [] and β.epsSuffix = [] and β.endState = β.startState.” A quiescent start state may still be followed by committed Σ-events, and those Σ-events may update state and create ε-suffix normalization. Also, a bucket may advance the clock, so the idle fixed-point conclusion uses NonClockStateEq rather than literal equality of startState and endState.

B4_finiteEpsilonDepth exposes the main-document B4 consequence through MainB4Bound: each process has a single finite D_P-style ε-stutter bound before a Σ-action or stable wait. ProcessEpsilonBound and IntraCycleNormalizationBound remain useful Lean-internal components, but they are not separate replacements for B4. The formalization must prove that the internal decomposition implies the single unified B4 finite-stutter obligation. QuiescentNormalization may still use IntraCycleNormalizationBound for bucket well-formedness and ε-quiescent cutpoint construction, but those bounds are applied per process by counting only the ε-steps owned by that process.

Bundle B-rules:
structure BRules (C : ComparisonInstance) where
B1_postDeterminism :
PostDeterministicSigmaTrace C

-- B2 is not post-only. It applies to any valid infinite execution of the
-- selected comparison side when that side participates in a progress/liveness
-- theorem.
B2_weakFairness :
∀ side τ,
InfiniteExecutionOfSide C side τ →
WeakFairness C side τ

-- B3 is likewise side-parametric. The P_push/P_pop obligations are required
-- for the reference-side Sys_ref execution as well as the post-side RTL_B
-- execution when the theorem uses those executions for liveness/deadlock
-- reasoning.
B3_channelProgress :
∀ side τ c,
InfiniteExecutionOfSide C side τ →
P_push C side τ c ∧ P_pop C side τ c

B4_finiteEpsilonDepth :
FiniteEpsilonDepth C
B5_quiescentNormalization :
QuiescentNormalization C
B6_idleStutterTimeDivergence :
IdleStutterTimeDivergence C

B2/B3 remain temporal assumptions usable by K proofs. B4/B5 also remain assumptions, but they are now constructive assumptions: they provide the finite ε-normalization facts needed to bound epsPrefix / epsSuffix lengths in CycleBucket and to prove bounded ε-chain lemmas such as Lemma I.1.

---

7. Channel semantics and E4

Define channel state over the selected Sys_ref boundary.

The primitive abstract occupancy is cycle-boundary, non-fall-through occupancy. It is indexed by the cycle, not by the full intra-cycle state:

capacity : C.Sys_ref.Channel → Nat
occAt : C.Sys_ref.Channel → Cycle → Nat

For convenience, one may define a state-indexed abbreviation only by projecting the state to its cycle:

occAtState : C.Sys_ref.Channel → State → Nat
occAtState c σ := occAt c σ.cycle

This abbreviation is not a second occupancy notion. In particular, occupancy is invariant under ε-steps and under same-cycle Σ-events:

occ_invariant_within_cycle :
∀ c σ_1 σ_2,
σ_1.cycle = σ_2.cycle →
occAtState c σ_1 = occAtState c σ_2

-- Domain for the one-cycle Appendix C request-at-issue obligation.
-- This domain includes every executed source-level message-operation instance,
-- including failed PushNB / PopNB polls that produce no committed Σ-event.
def BoundaryReqIssueDomainSide
(C : ComparisonInstance)
(side : Side)
(q : DynMsgInstance)
(σ : State) : Prop :=

∃ P,
q ∈ QExecSide C side P ∧
MessageOperationInstance q ∧
OwnsMessageEndpoint P q ∧
IssuedAtState C side q σ

-- Domain for persistent operation-attributive request/channel reasoning.
-- This is the domain used by ChannelEnabled, ChannelDisabled, Front, NextFront,
-- automatic flush, and WFG construction. Failed NB-only polls are not included
-- merely because they executed.
def BoundaryReqPersistentDomainSide
(C : ComparisonInstance)
(side : Side)
(q : DynMsgInstance)
(σ : State) : Prop :=
∃ P,
q ∈ QOmegaSide C side P ∧
MessageOperationInstance q ∧
OwnsMessageEndpoint P q

-- General BoundaryReq domain.
-- A request may be meaningful either because q is issuing in the current cycle
-- or because q is a persistent message-operation frontier instance.
def BoundaryReqDomainSide
(C : ComparisonInstance)
(side : Side)
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqIssueDomainSide C side q σ ∨
BoundaryReqPersistentDomainSide C side q σ

Executed failed non-blocking polls may belong to Qexec,P for replay and witness purposes while remaining absent from QOmega,P and Ω_P. Such polls do acquire the one-cycle BoundaryReq-at-issue obligation when they issue, via BoundaryReqIssueDomainSide. They do not acquire persistent BoundaryReqPersistentDomainSide, ChannelEnabled, ChannelDisabled, Front, NextFront, automatic-flush, or WFG obligations merely because they executed.

```
BoundaryReqSide :
ComparisonInstance → Side → DynMsgInstance → State → Prop

ChannelEnabledSide :
ComparisonInstance → Side → DynMsgInstance → State → Prop

def ChannelDisabledSide
(C : ComparisonInstance)
(side : Side)
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqPersistentDomainSide C side q σ ∧
BoundaryReqSide C side q σ ∧
¬ ChannelEnabledSide C side q σ
```

For post-side automatic-flush and issue/commit formulas, use the following abbreviations:

```
def BoundaryReqIssueDomain
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqIssueDomainSide C .post q σ

def BoundaryReqPersistentDomain
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqPersistentDomainSide C .post q σ

def BoundaryReqDomain
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqDomainSide C .post q σ

def BoundaryReq
(C : ComparisonInstance)
```

```
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqSide C .post q σ

def ChannelEnabled
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
ChannelEnabledSide C .post q σ

def ChannelDisabled
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
ChannelDisabledSide C .post q σ
```

BoundaryReqSide, ChannelEnabledSide, and ChannelDisabledSide are the canonical side-indexed predicates. The shorter BoundaryReq / ChannelEnabled / ChannelDisabled forms are post-side abbreviations used only where the section is already explicitly about RTL_B / post-side behavior.

BoundaryReqSide is used under BoundaryReqDomainSide. This includes both one-cycle issue requests, via BoundaryReqIssueDomainSide, and persistent frontier/channel requests, via BoundaryReqPersistentDomainSide.

ChannelEnabledSide and ChannelDisabledSide are used only under BoundaryReqPersistentDomainSide or its post-side abbreviation BoundaryReqPersistentDomain. Thus failed non-blocking polls in Q_exec,P \ Ω_P may satisfy the Appendix C one-cycle request-at-issue rule, but they still do not become message-operation frontier items and do not acquire ChannelEnabled, ChannelDisabled, Front, NextFront, automatic-flush, or WFG obligations merely because they executed.

ChannelEnabled is allowed to inspect the settled boundary-request state σ for persistent message-operation frontier instances, but its capacity/occupancy component must use occAt q.channel σ.cycle. Thus E4 enablement is evaluated against the beginning-of-cycle abstract occupancy, with no same-cycle fall-through through ε-settling or same-cycle Σ-events.

Capacity cases:

inductive CapacityKind
| rendezvous -- B(c) = 0
| buffered -- B(c) > 0

Expected lemmas:

BufferedPushEnabled
BufferedPopEnabled
RendezvousEnabled
FIFOOrderPreservation
NoDropNoDuplicate
OccupancyUpdatePush
OccupancyUpdatePop

E4 should be parameterized by Sys_ref, so in elaborated cases it ranges over explicit elaborated channels, not merely outer channels.

---

8. Appendix Q elaboration interface

Appendix Q should be introduced early. In the updated document, elaboration is not a late proof afterthought: in elaborated cases, Sys_ref = Sys_B^elab(E), and all references to B, occ, BoundaryReq, ChannelEnabled/Disabled, Front_P, and WFG are interpreted over the explicit channels of the elaborated model.

As in the ordinary E4 semantics, elaborated-channel occupancy is cycle-boundary, non-fall-through occupancy. Thus occ_E(d,t) and A_E(c,t) are indexed by cycle t. State-indexed forms such as occ_E(d,σ) or A_E(c,σ), when used informally at ε-quiescent cutpoints, are only abbreviations for occ_E(d,σ.cycle) and A_E(c,σ.cycle).

Chan_outer should be a parameter fixed by the chosen outer Sys_B, not a field chosen internally by the elaboration package.

The elaboration projection is intentionally multi-valued. A proof-internal elaborated channel may contribute to no outer channel, one outer channel, or several outer channels. Separately, a proof-internal token/event may account for multiple outer Σ_DUT-visible channel effects in bundled or joined cases.

structure ElaborationPackage

```
(Sys_B : System)
(Chan_outer : Type)
(Σ_DUT : Type) where
Sys_elab : System
Chan_elab : Type
finite_chan_elab : Fintype Chan_elab
ElabToken : Type
B_E : Chan_elab → Nat
occ_E : Chan_elab → Cycle → Nat

-- Channel-level accounting support.
-- Π_elab d is the set of outer channels whose aggregate accounting may depend
-- on elaborated channel d. The empty set represents a proof-internal channel
-- that is hidden from the outer DUT/testbench boundary.
Pi_elab : Chan_elab → Set Chan_outer

-- Token/event-level accounting support.
-- Π_token_elab tok is the set of outer channels accounted for by one
-- proof-internal token/event. This may contain multiple outer channels for
-- bundled or joined work items.
Pi_token_elab : ElabToken → Set Chan_outer
tokenOfSigma : Sys_elab.Sigma → Option ElabToken

-- Bucketed outer-projection of the elaborated proof-internal execution.
-- This replaces any flat Trace-level projection. The projection may hide
-- proof-internal staged/bundle/join events, but it must preserve the
-- cycle-bucket structure needed for ε-quiescent cutpoints, same-cycle atomic
-- units, and temporal B2/B3 reasoning.
piSigmaBucket :
CycleBucket Sys_elab.Sigma →
CycleBucket Σ_DUT

piSigmaFinite :
ExecutionBucketed Sys_elab.Sigma →
ExecutionBucketed Σ_DUT

piSigmaInfinite :
InfiniteExecutionBucketed Sys_elab.Sigma →
```

InfiniteExecutionBucketed Σ_DUT

piSigma_bucket_wf : Prop
piSigma_finite_agrees_with_bucket : Prop
piSigma_infinite_agrees_with_bucket : Prop

A_E : Chan_outer → Cycle → Nat
outer_bucketed_execution_preservation : Prop
occupancy_preservation : Prop
internal_refinement : Prop
no_hidden_cross_channel_disablement : Prop
wfg_visibility : Prop
combinational_wellformedness : Prop

For each outer channel c, the channel-level fiber of the elaboration is now understood as

$\Pi\_elab^{-1}(c) = \{ d : Chan\_elab \mid c \in Pi\_elab\ d \}$.

This replaces the single-valued Option-based fiber. The token-level projection Pi_token_elab is the more precise object used when one proof-internal staged/bundle/join token accounts for several outer Σ_DUT-visible channel effects. The bucketed projection piSigmaBucket, together with piSigmaFinite and piSigmaInfinite, and the occupancy/accounting map A_E must be consistent with both the channel-level support map Pi_elab and the token-level projection Pi_token_elab. In particular, elaboration projection is not a flat event-list projection: it preserves the cycle-bucket decomposition needed for ε-quiescent cutpoints, same-cycle atomic units, R2C fixed-point buckets, and B2/B3 temporal reasoning, while allowing proof-internal staged/bundle/join events to be hidden from the outer Σ_DUT view.

For convenience, the elaboration package may also define state-indexed abbreviations by projecting the state to its cycle:

occ_E_atState : Chan_elab → State → Nat
occ_E_atState d σ := occ_E d σ.cycle

A_E_atState : Chan_outer → State → Nat

A_E_atState c σ := A_E c σ.cycle

These abbreviations are not separate occupancy or accounting notions. They are invariant under ε-steps and same-cycle Σ-events:

occ_E_invariant_within_cycle :
∀ d σ_1 σ_2,
σ_1.cycle = σ_2.cycle →
occ_E_atState d σ_1 = occ_E_atState d σ_2

A_E_invariant_within_cycle :
∀ c σ_1 σ_2,
σ_1.cycle = σ_2.cycle →
A_E_atState c σ_1 = A_E_atState c σ_2

Reference choice:
inductive ReferenceKind
| ordinarySysB
| elaborated

structure SysRefChoice where
outerSysB : System
Chan_outer : Type
kind : ReferenceKind
elab? : Option (ElaborationPackage outerSysB Chan_outer Σ_DUT)
Sys_ref : System
sys_ref_matches_kind : Prop

For ordinarySysB, sys_ref_matches_kind entails Sys_ref = outerSysB.
For elaborated, sys_ref_matches_kind entails that there exists an elaboration package E with elab? = some E and Sys_ref = E.Sys_elab.

When SysRefChoice is embedded in a ComparisonInstance C, the field C.sys_ref_consistent additionally requires C.Sys_ref = C.refChoice.Sys_ref. Thus SysRefChoice records the construction/justification of the selected reference system, while ComparisonInstance.Sys_ref is the single reference system used by the theorem statements. There is no freedom for these two references to diverge.

This also handles sync-boundary epoch gates: if proof-relevant withholding exists, it must be represented by explicit gate channels in Sys_B^elab(E), not by hidden gating on an otherwise E4-enabled outer channel. The earlier review correctly flagged this as a point that must be explicit in the Lean strategy.

Locality note. Although ElaborationPackage is presented as a single structure parameterizing the theorem instance, its components (Sys_elab, Chan_elab, Π_elab, Π_token,elab, π_Σ,E, A_E) are constructed locally per process and per channel boundary. The channel-level projection Π_elab is multi-valued, and the token-level projection Π_token,elab is allowed to map one proof-internal staged/bundle/join token to multiple outer Σ_DUT-visible channels. Each per-process elaboration is determined by that process's local channel structure (number of inputs, sync-boundary discipline, pipelining configuration) and is independent of the elaboration chosen for any other process. The system-level ElaborationPackage is the disjoint composition of these per-process pieces. Treating E as a theorem-instance parameter is the right Lean encoding of "the verifier has chosen per-process elaborations and the proof takes them as given"; it is not a claim that the verifier must reason globally to construct E.

---

9. R-rules, E-rules, and modeling contracts

The R-rule bundle should include only the document’s actual R-rules plus an optional Lean-side array-rule pointer if desired. It should not include a fictional R5_syncBoundaryNoCrossing; sync-boundary crossing is E5. The document explicitly labels “Messages cannot cross their immediate synchronization boundary” as E5.

However, R5 itself is a real R-rule: R5 (Channel capacity semantics; Sys vs. Sys_B). In the Lean strategy, R5 should record the chosen source/reference-model capacity convention: raw Sys uses rendezvous capacity, while Sys_B / Sys_ref uses the selected capacity B(c), with B(c)=0 interpreted as rendezvous and B(c)>0 interpreted as cycle-boundary, non-fall-through bounded FIFO. The concrete enablement and occupancy-update obligations remain the E4 channel-capacity semantics; R5 records which source/reference capacity semantics are being used by the selected comparison instance.

```
def R2CCombinationalFixedPointRule
(C : ComparisonInstance) : Prop :=
SequentialOnly C ∨
∃ hR2C : R2C_WF C,
```

```
R2CEventsObservedOnlyAtFixedPoint C hR2C

def R5ChannelCapacitySemantics
(C : ComparisonInstance) : Prop :=
SysRefUsesSelectedCapacitySemantics C ∧
∀ c,
(capacity C c = 0 → RendezvousChannel C c) ∧
(0 < capacity C c → BufferedChannel C c)

structure RRules (C : ComparisonInstance) where
R1_syncOrder : Prop
R2_signalAnchorSequential : Prop
R2C_combinationalFixedPoint :
R2CCombinationalFixedPointRule C
R3_syncSemantics : Prop
R4_messageIssueOrder : Prop
R5_channelCapacitySemantics :
R5ChannelCapacitySemantics C
```

R4_messageIssueOrder is deliberately named only as an issue-order rule. It records the source-induced no-reverse discipline for message-operation issue. It does not impose a separate commit-order rule; commit timing is constrained by E4 channel availability, FIFO/rendezvous semantics, backpressure, and the chosen Sys_ref / RTL_B comparison instance.

E-rules:

```
structure ERules (C : ComparisonInstance) where
  E1_sync_order_preservation : Prop
  E2_signal_anchor_preservation : Prop
  E3_message_order_preservation : Prop
  E4_channel_capacity_semantics : Prop
  E5_sync_boundary_preservation : Prop
```

Array and direct-input conditions live in:

```
structure ModelingContracts (C : ComparisonInstance) where
  directInputs : DirectInputContract C
  arrays       : ArrayAccessMappingRules C
```

This avoids treating environment/design-side contracts inconsistently.

10. Array access mapping

External arrays are not merely internal Store. The document states that external arrays are mapped onto IO operations by an array access mapping layer, and that array accesses before a synchronization call must commit no later than the sync commit, while accesses after the sync must commit strictly after it. It also permits transformations such as caching, merging, splitting, and reordering when allowed by the mapping layer.

Define:

```
inductive MemoryEvent
| read  (array : ArrayId) (addr : Addr) (value : Value)
| write (array : ArrayId) (addr : Addr) (value : Value)

structure ArrayAccessMapping where
 sourceArrayAccess : SourceArrayAccess → List CoreIOEvent
 commitTime       : SourceArrayAccess → Cycle
 fixedLatency     : SourceArrayAccess → Nat

 conflictFreeReorderAllowed :
  SourceArrayAccess → SourceArrayAccess → Prop

 transformedEquivalent :
  SourceArrayAccess → List SourceArrayAccess → Prop
```

Rules:

Array accesses are constrained by the same dynamic synchronization intervals as the ordinary source-order frontier, but SourceArrayAccess values are not themselves members of prefS(s) or suffS(s). The array-access mapping layer must therefore provide an explicit bridge from source array accesses to the dynamic frontier/synchronization interval structure.

```
structure ArraySyncIntervalBridge
(C : ComparisonInstance) where
arrayFrontierItemOf :
SourceArrayAccess → Option FrontierItem

array_item_in_dynamic_universe :
```

∀ a x,
arrayFrontierItemOf a = some x →
∃ P W,
x ∈ (C.witness.dynamicUniverse P W).Ω

array_before_sync_matches_prefS :
∀ a s x,
arrayFrontierItemOf a = some x →
ArrayAccessBeforeSync C a s ↔
FrontierItemInPrefS C x s

array_after_sync_matches_suffS :
∀ a s x,
arrayFrontierItemOf a = some x →
ArrayAccessAfterSync C a s ↔
FrontierItemInSuffS C x s

array_commit_cycle_is_mapping_commit_cycle :
∀ a,
ArrayCommitCycle C a = ArrayCommitTime a

structure ArrayAccessMappingRules (C : ComparisonInstance) where
syncIntervalBridge :
ArraySyncIntervalBridge C

beforeSyncCommitsNoLater :
∀ a s,
ArrayAccessBeforeSync C a s →
ArrayCommitTime a ≤ SyncCommitTime s

afterSyncCommitsAfter :
∀ a s,
ArrayAccessAfterSync C a s →
SyncCommitTime s < ArrayCommitTime a

issueConsistentWithFixedLatency :
∀ a,
ArrayCommitTime a =

ArrayIssueTime a + ArrayFixedLatency a

transformedOpsRespectMapping :
∀ a transformed,
TransformedFrom a transformed →
MappingEquivalent a transformed

conflictFreeReorderingOnly :
  ∀ a_1 a_2,
   Reordered a_1 a_2 →
   conflictFreeReorderAllowed a_1 a_2

For early milestones, it is acceptable to assume:

AllExternalArrayAccessesAlreadyMappedToCoreIO C

but the full proof strategy should include array mapping.

---

11. R2C combinational fixed point

ComparisonInstance now has:
combModel? : Option CombComposition

R2C should use that field when combinational processes are in scope. The document says combinational Read/Write events occur in the observable bucket corresponding to the cycle's ε-quiescent combinational fixed point, and that combinational signal IO is well formed only when the relevant combinational network is deterministic and reaches a unique ε-fixed point.

The Lean model should preserve both facts:
1. R2C observes one composed fixed-point valuation μ_t for the cycle.
2. That composed valuation is obtained from explicit local combinational process components, rather than from an unexplained monolithic global object.

Define:
structure LocalCombNetwork where
proc : Process
combStepLocal : Store → Store → Prop
reads : Finset Signal

writes : Finset Signal
localDeterministic : Prop

structure CombNetwork where
combStep : Store → Store → Prop
deterministic : Prop

structure CombComposition where
components :
Process → Option LocalCombNetwork

composed :
CombNetwork

component_processes_are_combinational :
∀ P N,
components P = some N →
CombinationalProcess P

component_owner_matches_process :
∀ P N,
components P = some N →
N.proc = P

local_step_embeds_in_composed_step :
∀ P N σ σ',
components P = some N →
N.combStepLocal σ σ' →
composed.combStep σ σ'

composed_step_generated_by_components :
∀ σ σ',
composed.combStep σ σ' →
∃ P N,
components P = some N ∧
N.combStepLocal σ σ'

resolved_driver_or_disjoint_write_policy :

∀ P_1 P_2 N_1 N_2 sig,
components P_1 = some N_1 →
components P_2 = some N_2 →
sig ∈ N_1.writes →
sig ∈ N_2.writes →
$P_1 \neq P_2$ →
ResolvedMultiDriverPolicy composed sig

composed_deterministic_from_components :
(∀ P N, components P = some N → N.localDeterministic) →
composed.deterministic

The scheduling document's R2C rule requires deterministic ε-settling to a unique observable fixed point. It does not globally require structural combinational acyclicity for every admissible combinational network. Pure combinational oscillation, ambiguous unresolved multi-driver behavior, or non-convergent ε-behavior are excluded through the unique-fixed-point obligation below.

Structural acyclicity may still be imposed as a separate well-formedness requirement for specific elaboration/coupler constructions in Appendix Q, but it is not a universal requirement of LocalCombNetwork, CombNetwork, or CombComposition itself.

def CombQuiescent (N : CombNetwork) (σ : State) : Prop :=
¬ ∃ σ', N.combStep σ.store σ'.store

def CombFixedPointAtCycle
(N : CombNetwork)
(t : Cycle)
(μ : Store) : Prop :=
ReachesByCombSteps N t μ ∧
IsFixedPoint N μ

def ComponentParticipatesInCycle
(C : ComparisonInstance)
(M : CombComposition)
(P : Process)
(t : Cycle) : Prop :=

∃ N,
M.components P = some N ∧
CombinationalProcessActiveAt C P t

Well-formedness:
structure R2C_WF (C : ComparisonInstance) where
composition_present :
C.combModel?.isSome

uniqueComposedFixedPoint :
∀ t M,
C.combModel? = some M →
∃! μ, CombFixedPointAtCycle M.composed t μ

component_events_use_composed_fixed_point :
∀ P e t μ M,
C.combModel? = some M →
ComponentParticipatesInCycle C M P t →
CombReadWriteEventOfProcess C P e →
e ∈ BucketSigmaEvents C t →
CombFixedPointAtCycle M.composed t μ →
EventReadsWritesStore e μ

all_comb_events_have_component :
∀ P e t M,
C.combModel? = some M →
CombReadWriteEventOfProcess C P e →
e ∈ BucketSigmaEvents C t →
∃ N,
M.components P = some N

def R2CEventsObservedOnlyAtFixedPoint
(C : ComparisonInstance)
(hR2C : R2C_WF C) : Prop :=
∀ P e t,
CombReadWriteEventOfProcess C P e →
e ∈ BucketSigmaEvents C t →
∃ M μ,

C.combModel? = some M ∧
ComponentParticipatesInCycle C M P t ∧
CombFixedPointAtCycle M.composed t μ ∧
EventReadsWritesStore e μ

This formulation keeps the semantic fixed point global where R2C needs it, but makes the proof obligation modular: each combinational process contributes a local component, and the composed network used for μ_t is justified by the CombComposition fields. Thus R2C fixed-point equality remains a system-level statement, while conformance remains checkable process-by-process plus an explicit composition/driver-resolution obligation.

If combinational processes are out of scope for an early milestone, use:
SequentialOnly C
and postpone R2C_WF.

---

12. Issue/Commit well-formedness

Issue and Commit should be relational plus existence/uniqueness, not necessarily computable total functions.

The Issue/Commit layer is side-parametric and uses the same bucketed execution carrier as the rest of the strategy. It must not introduce a separate flat Trace type.

```
def SystemOfSide
(C : ComparisonInstance) : Side → System
| Side.ref => C.Sys_ref
| Side.post => C.RTL_B

abbrev SidePrefix
(C : ComparisonInstance)
(side : Side) : Type :=
Prefix (SystemOfSide C side)

abbrev SideInfiniteExecution
(C : ComparisonInstance)
(side : Side) : Type :=
```

InfiniteExecutionBucketed C.Sigma

IssueAt :
(C : ComparisonInstance) →
(side : Side) →
SidePrefix C side →
DynMsgInstance →
Cycle →
Prop

CommitAt :
(C : ComparisonInstance) →
(side : Side) →
SidePrefix C side →
DynMsgInstance →
Cycle →
Payload →
Prop

def CommitAtCycle
(C : ComparisonInstance)
(side : Side)
(τ : SidePrefix C side)
(q : DynMsgInstance)
(t : Cycle) : Prop :=
∃ v, CommitAt C side τ q t v

Well-formedness:

def BoundaryRequestAtIssue
(C : ComparisonInstance) : Prop :=
∀ side τ q tIssue σ,
IssuedInPrefix C side τ q →
IssueAt C side τ q tIssue →
StateAtCycleInPrefix C side τ tIssue σ →
BoundaryReqIssueDomainSide C side q σ ∧
BoundaryReqSide C side q σ ∧
((q.kind = MsgKind.Push ∨ q.kind = MsgKind.PushNB) →
PayloadAtState C side q σ = PayloadAtIssue C side τ q tIssue)

```
def BlockingBoundaryRequestPersistence
(C : ComparisonInstance) : Prop :=
∀ side τ q tIssue t σ,
IssuedInPrefix C side τ q →
IssueAt C side τ q tIssue →
StateAtCycleInPrefix C side τ t σ →
tIssue ≤ t →
¬ CommitsBefore C side τ q t →
Blocking q →
BoundaryReqPersistentDomainSide C side q σ ∧
BoundaryReqSide C side q σ ∧
(q.kind = MsgKind.Push →
PayloadAtState C side q σ = PayloadAtIssue C side τ q tIssue)

def BoundaryRequestSoundness
(C : ComparisonInstance) : Prop :=
BoundaryRequestAtIssue C ∧
BlockingBoundaryRequestPersistence C

def StableAttribution
(C : ComparisonInstance) : Prop :=
∀ side τ q₀ q₁ t σt σnext,
SameEndpointDirection q₀ q₁ →
q₀ ≠ q₁ →
Blocking q₀ →
StateAtCycleInPrefix C side τ t σt →
StateAtCycleInPrefix C side τ (t + 1) σnext →
BoundaryReqDomainSide C side q₀ σt →
BoundaryReqSide C side q₀ σt →
BoundaryReqDomainSide C side q₁ σnext →
BoundaryReqSide C side q₁ σnext →
¬ EndpointCommitAt C side τ q₀.endpoint t →
¬ EndpointCommitAt C side τ q₁.endpoint t →
False
```

BoundaryRequestSoundness has two parts. BoundaryRequestAtIssue applies to

every issued source-level message-operation instance q, including PushNB and PopNB instances that fail and therefore produce no committed Σ-event. Such a non-blocking poll must still assert the appropriate endpoint request in its issue cycle. BlockingBoundaryRequestPersistence is the additional non-withdrawal rule for blocking Push/Pop instances: after issue and before commit, the same dynamic operation instance keeps its request asserted, and a Push keeps its payload stable.

StableAttribution applies when the earlier outstanding request q_0 is a blocking Push/Pop. In that case, attribution cannot change to any distinct same-endpoint operation before a commit on that endpoint direction, whether the later candidate q_1 is blocking or non-blocking. For non-blocking PushNB / PopNB calls, request assertion at the issue cycle does not by itself imply persistence beyond that issue cycle, and continuous assertion of the endpoint request signal does not by itself collapse later executed non-blocking call instances into the same dynamic instance q. Repeated executed NB polls remain distinct source-level dynamic instances unless the witness explicitly identifies the same still-outstanding request instance.

```
-- Dynamic source interval between synchronization anchors.
-- This is witness-specific: only executed dynamic instances in the selected
-- witness universe are classified. Untaken branches and unexecuted loop
-- iterations have no SyncIntervalId.

structure SyncInterval where
proc : Process
pred? : Option SyncCall
succ? : Option SyncCall
intervalId : Nat

def SyncIntervalId
(C : ComparisonInstance)
(W : WitnessBundle C)
(q : DynMsgInstance) : Option SyncInterval :=
DynamicSourceIntervalOf C W q

structure SyncIntervalWF
```

(C : ComparisonInstance)
(W : WitnessBundle C) where
interval_defined_for_frontier_messages :
∀ P q,
q ∈ (C.witness.dynamicUniverse P W).Q_Ω →
∃ I, SyncIntervalId C W q = some I

same_interval_iff_same_anchors :
∀ q_0 q_1 I_0 I_1,
SyncIntervalId C W q_0 = some I_0 →
SyncIntervalId C W q_1 = some I_1 →
I_0 = I_1 ↔
I_0.proc = I_1.proc ∧
I_0.pred? = I_1.pred? ∧
I_0.succ? = I_1.succ? ∧
I_0.intervalId = I_1.intervalId

pref_suff_consistent :
∀ s q,
q ∈ prefS s ∨ q ∈ suffS s →
∃ I, SyncIntervalId C W q = some I

def SameSyncInterval
(C : ComparisonInstance)
(W : WitnessBundle C)
(q_0 q_1 : DynMsgInstance) : Prop :=
∃ I,
SyncIntervalId C W q_0 = some I ∧
SyncIntervalId C W q_1 = some I

def SourceIssuesBackToBackAfterCommit
(C : ComparisonInstance)
(W : WitnessBundle C)
(side : Side)
(τ : SidePrefix C side)
(q_0 q_1 : DynMsgInstance)
(t : Cycle) : Prop :=

SameEndpointDirection q_0 q_1 ∧
SameSyncInterval C W q_0 q_1 ∧
CommitAtCycle C side τ q_0 t ∧
NextSourceMessageOpOnEndpoint C q_0 q_1 ∧
IssuedInPrefix C side τ q_1 ∧
SourceControlReachesAndIssuesAt C W side τ q_1 (t + 1)

def BackToBackIssueWF
(C : ComparisonInstance)
(W : WitnessBundle C) : Prop :=
∀ side τ q_0 q_1 t,
SourceIssuesBackToBackAfterCommit C W side τ q_0 q_1 t →
EndpointRequestContinuouslyAsserted C side τ q_0.endpoint t (t + 1) →
IssueAt C side τ q_1 (t + 1)

The SameSyncInterval component of SourceIssuesBackToBackAfterCommit prevents the back-to-back issuance rule from carrying a continuously asserted endpoint request across an explicit wait(), SyncChannel, ac_sync, or other synchronization boundary. Cross-boundary issue timing is governed instead by SyncBoundaryIssueDiscipline / E5.

The continuous physical endpoint assertion is not what causes q_1 to issue. It only explains why the boundary signal may remain asserted across the q_0/q_1 transition without a visible deassertion bubble. The source-level cause is the separate SourceControlReachesAndIssuesAt premise: q_1 has actually been reached and issued as the next same-endpoint operation in the same synchronization interval. Without that premise, a held valid/ready signal could be incorrectly used to infer issuance of a source operation that control flow has not yet reached.

def SyncBoundaryIssueDiscipline
(C : ComparisonInstance) : Prop :=
∀ side τ q_0 q_1 s tSync t_0 t_1,
q_0 ∈ prefS s →
q_1 ∈ suffS s →
SyncCommitAt C side τ s tSync →
IssueAt C side τ q_0 t_0 →

IssueAt C side τ q_1 t_1 →
t_0 ≤ tSync ∧ tSync < t_1

def NBDynamicInstanceDiscipline
(C : ComparisonInstance) : Prop :=
∀ P q_0 q_1,
IsNB q_0 →
IsNB q_1 →
ExecutedInProcess C P q_0 →
ExecutedInProcess C P q_1 →
SameSourceCallSiteButDifferentExecution q_0 q_1 →
q_0 ≠ q_1

structure IssueCommitWF
(C : ComparisonInstance)
(W : WitnessBundle C) where
issue_exists :
∀ side τ q,
IssuedInPrefix C side τ q →
∃ t, IssueAt C side τ q t
issue_unique :
∀ side τ q t_1 t_2,
IssueAt C side τ q t_1 →
IssueAt C side τ q t_2 →
t_1 = t_2
commit_exists_unique :
∀ side τ q,
CommitsInPrefix C side τ q →
∃! tp : Cycle × Payload,
CommitAt C side τ q tp.1 tp.2
boundary_request_soundness :
BoundaryRequestSoundness C
stable_attribution :
StableAttribution C
back_to_back_issue :
BackToBackIssueWF C W
sync_boundary_discipline :
SyncBoundaryIssueDiscipline C

nb_dynamic_instance_discipline :
NBDynamicInstanceDiscipline C

Derived timestamp:
noncomputable def IssueTime
(hWF : IssueCommitWF C W)
(h : IssuedInPrefix C side τ q) : Cycle :=
Classical.choose (hWF.issue_exists side τ q h)

This preserves Appendix C's use of Issue(q) as an auxiliary timestamp while avoiding an unnecessary computability requirement.

---

13. Automatic flush and sync-boundary policy

Define:
Pending P σ
Front P σ
NextFront P σ
BlockingMsgNextFront P σ
Done P σ
Remain P σ

Automatic flush must be stated over the same dynamic universe and source-order objects used by Appendix I/J/K. In particular, it must not be an opaque Prop bundle.

Section 13 reasons primarily about FrontierItem values, while Section 7's BoundaryReqSide / ChannelEnabledSide / ChannelDisabledSide predicates are operation-attributive predicates over DynMsgInstance values. Therefore Section 13 uses the following item-level wrappers. These wrappers are not new semantic predicates; they merely read a message-operation frontier item through its owning dynamic message instance.

def MsgOfFrontierItemSide
(C : ComparisonInstance)
(side : Side)

```
(x : FrontierItem) : Option DynMsgInstance :=
MsgInstanceOfFrontierItem C x.proc x

def BoundaryReqItemSide
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem)
(σ : State) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
BoundaryReqPersistentDomainSide C side q σ ∧
BoundaryReqSide C side q σ

def ChannelEnabledItemSide
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem)
(σ : State) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
ChannelEnabledSide C side q σ

def ChannelDisabledItemSide
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem)
(σ : State) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
ChannelDisabledSide C side q σ

def PayloadAtFrontierItemState
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem)
(σ : State) : Payload :=
PayloadAtState C side (Classical.choose (MsgOfFrontierItemSide_isSome C side x)) σ
```

```
def FrontierItemIssuedAt
(C : ComparisonInstance)
(side : Side)
(τ : SidePrefix C side)
(x : FrontierItem)
(t : Cycle) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
IssueAt C side τ q t

def FrontierItemCommitsBeforeCycle
(C : ComparisonInstance)
(side : Side)
(τ : SidePrefix C side)
(x : FrontierItem)
(t : Cycle) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
CommitsBefore C side τ q t

def FrontierItemCommitsBetween
(C : ComparisonInstance)
(side : Side)
(τ : SidePrefix C side)
(x : FrontierItem)
(t₀ t : Cycle) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
CommitsBetween C side τ q t₀ t

def FrontierItemCommitsAtCycle
(C : ComparisonInstance)
(side : Side)
(τ : SidePrefix C side)
(x : FrontierItem)
(t : Cycle) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
```

CommitAtCycle C side τ q t

def FrontierBlocked
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State) : Prop :=
Front C side P σ ≠ ∅ ∧
∀ x,
x ∈ Front C side P σ →
BlockingMessageFrontierItem x ∧
BoundaryReqItemSide C side x σ ∧
ChannelDisabledItemSide C side x σ

def LaterThanBlockedFrontier
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State)
(y : FrontierItem) : Prop :=
∃ x,
x ∈ Front C side P σ ∧
SourceOrderLt C side P x y

def FrontierRemainsBlockedOver
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side)
(t_0 t : Cycle) : Prop :=
∀ n σn,
t_0 ≤ n →
n ≤ t →
StateAtCycleInPrefix C side τ n σn →
FrontierBlocked C side P σn

def NoNewLaterWorkWhileBlocked

(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) : Prop :=
∀ t_0 t $σ_0$ y,
StateAtCycleInPrefix C side τ t_0 $σ_0$ →
FrontierBlocked C side P $σ_0$ →
FrontierRemainsBlockedOver C side P τ t_0 t →
LaterThanBlockedFrontier C side P $σ_0$ y →
¬ FrontierItemIssuedAt C side τ y t

def NoWithdrawalWhileBlocked
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) : Prop :=
∀ t_0 t $σ_0$ σt x,
StateAtCycleInPrefix C side τ t_0 $σ_0$ →
StateAtCycleInPrefix C side τ t σt →
t_0 ≤ t →
x ∈ Front C side P $σ_0$ →
BoundaryReqItemSide C side x $σ_0$ →
¬ FrontierItemCommitsBetween C side τ x t_0 t →
BoundaryReqItemSide C side x σt ∧
(IsPushFrontierItem x →
PayloadAtFrontierItemState C side x σt =
PayloadAtFrontierItemState C side x $σ_0$)

def FrontierMinimality
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) : Prop :=
∀ t σ x y,
StateAtCycleInPrefix C side τ t σ →
x ∈ Front C side P σ →
y ∈ Remain C side P σ →
SourceOrderLt C side P y x →

False

The next predicates separate two obligations that should not be conflated.

First, there is the Kε / A11-style WFG-inertness obligation. It applies only to intervals that contain no observable Σ-commit. It says that pure internal / ε-only movement while a process remains blocked at the same frontier cannot create a new wait-for cause.

Second, there is the active drain-only automatic-flush discipline. It permits already-issued downstream work to commit while the process is blocked at a minimal frontier. Such drain commits may be observable Σ-events, so they must not be excluded by an InternalOrEpsilonOnlyBetween premise. The discipline is instead that no new later source work is issued past the blocked frontier, already asserted requests are not withdrawn before commit, and any observable commit owned by the blocked process during the blocked interval is attributable to an already-issued drainable downstream request.

```
def WFGInertForProcess
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ₀ σt : State) : Prop :=
∀ Q,
WFGEdge C side σ₀ P Q ↔ WFGEdge C side σt P Q

def NoNewBlockingFrontierCause
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ₀ σt : State) : Prop :=
∀ x,
x ∈ Front C side P σt →
BlockingMessageFrontierItem x →
ChannelDisabledItemSide C side x σt →
∃ y,
y ∈ Front C side P σ₀ ∧
```

BlockingMessageFrontierItem y ∧
SameBlockingCause C side P y x

def WFGInertUnderInternalOnlyBetween
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) : Prop :=
∀ t_0 t σ_0 σt,
StateAtCycleInPrefix C side τ t_0 σ_0 →
StateAtCycleInPrefix C side τ t σt →
$t_0 \le t$ →
FrontierBlocked C side P σ_0 →
FrontierRemainsBlockedOver C side P τ t_0 t →
InternalOrEpsilonOnlyBetween C side τ t_0 t →
WFGInertForProcess C side P σ_0 σt ∧
NoNewBlockingFrontierCause C side P σ_0 σt

def DrainableDownstreamRequest
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ_0 : State)
(x : FrontierItem) : Prop :=
AlreadyIssuedBeforeBlockedCutpoint C side P σ_0 x ∧
DownstreamOfBlockedFrontier C side P σ_0 x ∧
BlockingMessageFrontierItem x ∧
BoundaryReqItemSide C side x σ_0

def DrainOnlyCommitDiscipline
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) : Prop :=
∀ t_0 t σ_0 e,
StateAtCycleInPrefix C side τ t_0 σ_0 →
$t_0 \le t$ →
FrontierBlocked C side P σ_0 →

FrontierRemainsBlockedOver C side P τ t_0 t →
e ∈ SigmaEventsAtCycleInPrefix C side τ t →
EventOwnedByProcess C side P e →
ObservableCommitEvent e →
∃ x,
DrainableDownstreamRequest C side P σ_0 x ∧
EventCommitsFrontierItem C side x e ∧
¬ FrontierItemCommitsBeforeCycle C side τ x t_0 ∧
FrontierItemCommitsAtCycle C side τ x t

def DrainOnlyDownstreamProgress
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) : Prop :=
WFGInertUnderInternalOnlyBetween C side P τ ∧
DrainOnlyCommitDiscipline C side P τ

structure AutomaticFlush
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) where
block_point :
∀ t σ,
StateAtCycleInPrefix C side τ t σ →
FrontierBlocked C side P σ →
FlushBlockedPoint C side P τ t σ

no_new_later_work :
NoNewLaterWorkWhileBlocked C side P τ

no_withdrawal :
NoWithdrawalWhileBlocked C side P τ

frontier_minimality :
FrontierMinimality C side P τ

drain_only_downstream_progress :
DrainOnlyDownstreamProgress C side P τ

sync_boundary_policy :
SyncBoundaryPolicy C side P τ

The sync-boundary policy must explicitly split the ordinary and elaborated cases. In the elaborated case, it must also state the content of the withholding rule: producer-facing availability for capacity belonging only to suff_S(s) is unavailable while the synchronization call s is pending.

```
def SyncPendingAt
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side)
(s : SyncCall)
(t : Cycle) : Prop :=
SyncIssuedButNotCommitted C side P τ s t

def FutureEpochCapacityFor
(C : ComparisonInstance)
(side : Side)
(P : Process)
(s : SyncCall)
(gateChannel : Channel)
(q : DynMsgInstance) : Prop :=
q ∈ suffS_msg C side P s ∧
MessageOperationInstance q ∧
TransferWouldUseCapacityBehindGate C side P s gateChannel q

def SyncBoundaryGateCorrect
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side)
(E : ElaborationPackage C.refChoice.outerSysB C.refChoice.Chan_outer C.Sigma_DUT)
(gateChannel : Channel)
```

(s : SyncCall) : Prop :=
GateChannelOf E gateChannel ∧
∀ t σ q,
StateAtCycleInPrefix C side τ t σ →
SyncPendingAt C side P τ s t →
FutureEpochCapacityFor C side P s gateChannel q →
BoundaryReqPersistentDomainSide C side q σ →
BoundaryReqSide C side q σ →
ChannelDisabledSide C side q σ
Here suffS_msg C side P s is the message-operation-instance projection of suffS(s). The sync-boundary gate rule is stated over DynMsgInstance values because BoundaryReqSide, ChannelEnabledSide, and ChannelDisabledSide are obligations of attributed message-operation instances, not of arbitrary FrontierItem records.

inductive SyncBoundaryPolicy
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side)
| no_withholding_in_ordinary_SysB :
C.refChoice.kind = ReferenceKind.ordinarySysB →
NoProofRelevantSyncBoundaryWithholding C side P τ →
SyncBoundaryPolicy C side P τ
| explicit_gate_in_elab :
∀ E gateChannel s,
C.refChoice.kind = ReferenceKind.elaborated →
SyncBoundaryGateCorrect C side P τ E gateChannel s →
SyncBoundaryPolicy C side P τ

In the ordinary Sys_B case, the sync-boundary clause does not add hidden gating. It asserts that no proof-relevant sync-boundary withholding is present. If such withholding is needed, the theorem instance must use Sys_B^elab(E), where the withholding appears as ordinary ChannelDisabled behavior at an explicit sync-boundary / epoch-gate abstract channel.

---

14. Witness selector and Appendix L

The uploaded document's witness selector is a partial strategy over ε-quiescent source cutpoints and ε-modeled choices whose outcomes may affect later Σ-visible behavior; it must be causally available from the matched RTL prefix and global Σ-causal history.

Avoid circularity by defining it over the partial construction so far:

```
structure WitnessSelector (C : ComparisonInstance) where
 choose :
 (τ_post_prefix : Prefix C.RTL_B) →
 (τ_ref_so_far : Prefix C.Sys_ref) →
 (cp : ChoicePoint τ_ref_so_far) →
 Option ChoiceOutcome
 causal :
 ∀ τp τr cp out,
 choose τp τr cp = some out →
 DependsOnlyOnAvailablePrefix τp τr cp out
 consistent_with_post_prefix :
 ∀ τp τr cp out,
 choose τp τr cp = some out →
 ConsistentWithRTLPrefix τp τr cp out
```

Non-blocking witness mapping.
Failed PushNB / PopNB polls are executed dynamic instances in Q_exec but produce no Σ-event. Therefore the Post → Ref construction cannot reconstruct them from the Σ-trace alone. The witness bundle must carry an explicit partial dynamic-instance correspondence for such ε-modeled NB instances.

```
structure NBInstanceMap (C : ComparisonInstance) where
μ_Q :
DynMsgInstance → Option DynMsgInstance

maps_only_executed_nb :
∀ q_post q_ref,
μ_Q q_post = some q_ref →
ExecutedInPost C q_post ∧
ExecutedInRef C q_ref ∧
IsNB q_post ∧ IsNB q_ref

preserves_owner_and_interface :
```

∀ q_post q_ref,
μ_Q q_post = some q_ref →
q_post.proc = q_ref.proc ∧
q_post.channel = q_ref.channel ∧
q_post.kind = q_ref.kind

preserves_nb_outcome :
∀ q_post q_ref,
μ_Q q_post = some q_ref →
NBOutcomePost C q_post = NBOutcomeRef C q_ref

covers_control_relevant_failed_polls :
∀ q_post,
ExecutedInPost C q_post →
IsFailedNB q_post →
AffectsLaterSigmaOrFrontier C q_post →
∃ q_ref, μ_Q q_post = some q_ref

source_order_position_preserved :
∀ q_post q_ref,
μ_Q q_post = some q_ref →
SourceOrderKeyOf C q_post = SourceOrderKeyOf C q_ref

respects_source_order_slice :
∀ P q_1_post q_2_post q_1_ref q_2_ref,
μ_Q q_1_post = some q_1_ref →
μ_Q q_2_post = some q_2_ref →
SourceOrderLe C P (SourceOrderKeyOf C q_1_post) (SourceOrderKeyOf C q_2_post) ↔
SourceOrderLe C P (SourceOrderKeyOf C q_1_ref) (SourceOrderKeyOf C q_2_ref)

structure WitnessBundle (C : ComparisonInstance) where
selector : WitnessSelector C
nbMap : NBInstanceMap C

Witness compatibility:
def WC
(C : ComparisonInstance)
(W : WitnessBundle C) : Prop :=

(∀ constructionPrefix,
WitnessChoicesRespectMatchedPrefix C W.selector constructionPrefix) ∧
WitnessNBMapRespectsMatchedPrefix C W.nbMap ∧
WitnessChoicesAndNBMapAgree C W.selector W.nbMap

Thus WC is the Lean-side packaging of both witness selection and the μ_Q matching obligation. In particular, when E3/E5 reasoning or the Post → Ref construction refers to failed ε-modeled non-blocking call instances in Q_exec,P, the required μ_Q correspondence is supplied by W.nbMap as part of WC, not as a separate unrelated assumption.

Lemma L.2:
theorem L2_snooping_establishes_WC
(C : ComparisonInstance)
(W : WitnessBundle C)
(hSnoop : SnoopingWrapperAssumptions C W) :
WC C W

IdentityOrArbitraryNBWitnessBundle C is the witness bundle used in the NB-CF case. Its selector uses identity choices for ε-modeled non-blocking outcomes that are fixed by the matched prefix, and arbitrary choices for non-blocking outcomes that NB-CF proves cannot affect later Σ-visible control flow or NextFront behavior. Its nbMap maps matched executed NB instances to the corresponding same dynamic source-call instance when such a match is relevant, and is arbitrary on failed NB instances that NB-CF proves irrelevant.

def IdentityOrArbitraryNBWitnessBundle
(C : ComparisonInstance) : WitnessBundle C :=
{
selector := IdentityOrArbitraryNBSelector C,
nbMap := IdentityOrArbitraryNBMap C
}

Corollary L.3:
theorem L3_nb_cf_identity_witness
(C : ComparisonInstance)
(P : Process)

(hNBCF : NBCF C P) :
WC C (IdentityOrArbitraryNBWitnessBundle C)

---

15. Concrete ObsSigma and deterministic replay

Do not define ObsSigma as an informal least fixed point. Define it as a concrete record of the source-state slices required for replay.

structure ObsSigma
(C : ComparisonInstance)
(W : WitnessBundle C)
(P : Process)
(σ : State) where
control_pos : ControlPos P
relevant_vars : RelevantVarSlice C P σ
selected_eps_outcomes : SelectedOutcomeSlice C W.selector P σ
nb_witness_slice : NBWitnessSlice C W.nbMap P σ
msg_attribution_slice : MsgAttributionSlice C W P σ
pending_slice : PendingSlice C P σ
front_slice : FrontSlice C P σ
nextfront_slice : NextFrontSlice C P σ
boundary_req_slice : BoundaryReqSlice C P σ
channel_fact_slice : ChannelFactSlice C P σ
signal_anchor_slice : SignalAnchorSlice C P σ
array_mapping_slice : ArrayMappingSlice C P σ
direct_input_slice : DirectInputSlice C P σ

Here msg_attribution_slice records the Q_exec,P / Q_Ω,P attribution facts needed for E3/E5, Issue/Commit replay, WFG construction, and BoundaryReq ownership. It is broader than nb_witness_slice: nb_witness_slice records μ_Q matching for ε-modeled failed NB calls, while msg_attribution_slice records the source-level operation attribution of pending and frontier message-operation instances generally.

Then:
def SigmaRelevantEq C W P σ_1 σ_2 :=
ObsSigma C W P σ_1 = ObsSigma C W P σ_2

Replay theorem:
theorem IDR_deterministic_replay
(C : ComparisonInstance)
(W : WitnessBundle C)
(hB : BRules C)
(hR : RRules C)
(hE : ERules C)
(hIC : IssueCommitWF C W)
(hContracts : ModelingContracts C)
(hWC : WC C W)
(P : Process)
(σ_1 σ_2 : State)
(hObs : SigmaRelevantEq C W P σ_1 σ_2) :
SameNextSigmaRelevantBehavior C W P σ_1 σ_2

This same ObsSigma object should be reused by Lemma S1 and Corollary J.1.

---

16. Appendix I per-process construction

Bundle the assumptions:

structure IConstructionAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where
wellformed : C.WellFormed
bRules : BRules C
rRules : RRules C
eRules : ERules C
issueCommitWF : IssueCommitWF C W
implementation : ImplementationAssumptions C
witnessCompatible : WC C W
contracts : ModelingContracts C

Main theorems:
theorem I1_bounded_epsilon_closure
(C : ComparisonInstance)
(hB4 : C.assumptions.B.B4_finiteEpsilonDepth)
(hB5 : C.assumptions.B.B5_quiescentNormalization) :

...

theorem I2_channel_progress_instance
(C : ComparisonInstance)
(hB3 : C.assumptions.B.B3_channelProgress) :
...

theorem I3_finite_extension_step
(C : ComparisonInstance)
(W : WitnessBundle C)
(hI : IConstructionAssumptions C W) :
...

theorem I4_finite_prefix_construction
(C : ComparisonInstance)
(W : WitnessBundle C)
(hI : IConstructionAssumptions C W) :
...

theorem I5_infinite_limit_construction
(C : ComparisonInstance)
(W : WitnessBundle C)
(hI : IConstructionAssumptions C W) :
...

I3 should use IssueAt plus derived IssueTime, not an unconstrained guessed issue cycle.

---

17. Appendix J system-level correspondence

Define execution scopes:

def Exec_post (C : ComparisonInstance) (τ : Prefix C.RTL_B) : Prop :=
∃ τ∞,
Extends τ∞ τ ∧
InfinitePostExecution C τ∞ ∧
FairProgressSatisfying C τ∞

```
def Exec_ref (C : ComparisonInstance) (τ : Prefix C.Sys_ref) : Prop :=
∃ τ∞,
Extends τ∞ τ ∧
RefAdmissible C τ∞
```

Here FairProgressSatisfying includes the B2/B3 fairness/progress assumptions on infinite executions. B6 is not limited to deadlock cutpoints. It is the general time-divergence / idle-stutter convention for any reachable finite prefix/state at which the system has reached a fixed point: after no further Σ-action or internal ε-microstep is enabled under the same stimulus, the global clock may continue by infinite idle-stutter ticks. Each idle-stutter bucket has no sigmaEvents, performs no ε-prefix or ε-suffix work, and leaves the global state unchanged except for the clock index.

```
def IdleStutterBucket
(C : ComparisonInstance)
(side : Side)
(σ : State)
(β : CycleBucket C.Sigma) : Prop :=
β.startState = AdvanceClockOnly σ β.cycle ∧
β.sigmaEvents = ∅ ∧
β.epsPrefix = [] ∧
β.epsSuffix = [] ∧
β.endState = β.startState ∧
β.endQuiescent

def FixedPointState
(C : ComparisonInstance)
(side : Side)
(σ : State) : Prop :=
EpsQuiescent σ ∧
NoObservableActionEnabledAtState C side σ ∧
NoInternalEpsilonStepEnabledAtState C side σ

theorem B6_extend_reachable_prefix_by_idle_stutter
(C : ComparisonInstance)
(side : Side)
(π : Prefix (SystemOfSide C side))
```

```
(σ : State)
(hReach : TerminalCutpoint π σ)
(hFP : FixedPointState C side σ) :
∃ τ : InfiniteExecutionBucketed C.Sigma,
ExtendsPrefix C side τ π ∧
∀ n,
n ≥ PrefixLength π →
IdleStutterBucket C side σ (τ n) :=
by
-- General B6 theorem: any reachable finite execution prefix/state at a
-- fixed point is extendable to an infinite execution by appending idle
-- stutter ticks under the same stimulus.
sorry

theorem deadlock_cutpoint_in_Exec_post
(C : ComparisonInstance)
(π : Prefix C.RTL_B)
(σ : State)
(hReach : TerminalCutpoint π σ)
(hDL : InternalMPDeadlockAt C .post σ) :
∃ τ,
InfinitePostExecution C τ ∧
ExtendsPrefix C .post τ π :=
by
-- Deadlock cutpoints are a special case of
-- B6_extend_reachable_prefix_by_idle_stutter.
sorry

theorem finish_process_idle_padding
(C : ComparisonInstance)
(side : Side)
(P : Process)
(π : Prefix (SystemOfSide C side))
(σ : State)
(hReach : TerminalCutpoint π σ)
(hFin : ProcessFinishedAt C side P σ)
(hNoMoreP : NoFurtherProcessStepEnabled C side P σ) :
∃ τ : InfiniteExecutionBucketed C.Sigma,
```

ExtendsPrefix C side τ π Λ
∀ n,
n ≥ PrefixLength π →
ProcessIdleStuttered C side P (τ n) :=
by
-- FINISH_P remains a realized SyncAnchor for R2/E2 signal anchoring, but
-- after the process has terminated, the infinite execution is padded by
-- process-local idle stutter. This padding creates no new process-owned
-- Σ-events, no new BoundaryReq, no new Front/NextFront item, and no new
-- WFG edge for P.
sorry

The FINISH_P corollary is process-local: a finished process is padded by ε/idle-stutter while the global system may continue executing other processes. This padding must not be counted as additional B4/B5 internal progress by P, because it performs no enabled internal ε-step and emits no Σ-event.

Directed compatibility:
Do not define CompatibleP or CompatibleSys as equality of bucketed executions.

First define the observable units extracted from a bucketed execution. An observable unit may be: a single ordinary Σ-event; an explicitly atomic multi-endpoint unit, such as a rendezvous Push/Pop pair or a SyncChannel/ac_sync pair; or an R2C fixed-point observation unit. Independent same-cycle events are not ordered merely because they share a CycleBucket; any required order among observable units is imposed only by the global HappensBefore relation.

structure ObservableUnit (Σ : Type) where
events : Finset Σ
atomic : Prop
ObservableUnitsOf extracts Σ-visible events from the bucketed execution and groups them into comparison units as follows:
• an ordinary committed message, signal, or synchronization event may form a singleton observable unit;
• explicitly atomic multi-endpoint events, such as rendezvous Push/Pop pairs and SyncChannel/ac_sync handshakes, form one atomic observable unit and may not be split by CompatibleP or CompatibleSys;

• R2C combinational observations are taken from the ε-quiescent fixed-point bucket and grouped according to the R2C well-formedness relation;
• independent same-cycle events are unordered merely by virtue of being in the same CycleBucket;
• same-cycle events that are merely coincident in one implementation are not forced to remain same-cycle in the other implementation unless they are explicitly atomic or otherwise ordered by the global HappensBefore relation.

A CycleBucket records which Σ-events commit in a cycle. It does not record a same-cycle sequence or a same-cycle partial order. Any required causal dependency involving events in the same or different buckets is derived by the global HappensBeforeGenerator below. Atomic multi-endpoint units constrain grouping/splitting; they do not introduce an arbitrary intra-bucket linearization.

```
def ObservableUnitsOf
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side)) :
Set (ObservableUnit (C.Sigma)) :=
ExtractSigmaUnits
C side τ
(AtomicRendezvousUnits C side τ)
(AtomicSyncUnits C side τ)
(R2CFixedPointUnits C side τ)
(IndependentSameCycleUnits C side τ)
```

HappensBefore is the transitive closure of the order generators that the document requires CompatibleP / CompatibleSys to preserve:
• per-process E1 synchronization order;
• E2 signal-anchor order;
• E3 same-interface order and distinct-interface no-reverse order;
• E4 per-channel FIFO/rendezvous order and capacity-derived dependency order;
• E5 synchronization-boundary preservation;
• explicit atomic-unit grouping constraints;
• witness-selected NB/control dependencies that affect later Σ-visible behavior, NextFront, or boundary-request attribution;
• elaboration-projection constraints when Sys_ref = Sys_B^elab(E).

```
def HappensBefore
```

(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (C.Sigma)) : Prop :=
Relation.TransGen
(HappensBeforeGenerator C side τ)
u v
def HappensBeforeGenerator
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (C.Sigma)) : Prop :=
E1SyncOrder C side τ u v ∨
E2SignalAnchorOrder C side τ u v ∨
E3MessageOrder C side τ u v ∨
E4ChannelOrder C side τ u v ∨
E5SyncBoundaryOrder C side τ u v ∨
AtomicUnitOrder C side τ u v ∨
WitnessControlOrder C side τ u v ∨
ElaborationProjectionOrder C side τ u v

CompatibleP : ComparisonInstance → Process → Prefix C.Sys_ref → Prefix C.RTL_B → Prop

CompatibleP C P τ_ref τ_post means:
• the P-local observable units of τ_ref and τ_post are related by a value-label-preserving bijection;
• explicitly atomic units are mapped to explicitly atomic units and are not split;
• the mapping preserves the P-local happens-before constraints induced by E1, E2, E3, E5, R2/R2C, and the selected witness/request-attribution facts;
• it does not require independent P-local units to occupy the same cycle bucket in both executions.

CompatibleSys : ComparisonInstance → Prefix C.Sys_ref → Prefix C.RTL_B → Prop

CompatibleSys C τ_ref τ_post means:
• ∀ P, CompatibleP C P τ_ref τ_post;
• all explicit abstract channels in the chosen Sys_ref / RTL_B comparison satisfy

the applicable E4 channel semantics at the agreed abstract boundaries;
• the system-level observable-unit mapping preserves the mandated global happens-before / $\prec_J$ constraints and value labels;
• explicitly atomic multi-endpoint units remain atomic on both sides;
• no equality of CycleBucket lists, no equality of bucket cycle numbers, and no bucket-by-bucket equality of sigmaEvents is required.

Thus the bijection used by CompatibleSys is over ObservableUnitsOf, and its ordering obligation is preservation of HappensBefore. Independent observable units may be placed in different cycle buckets or grouped differently in Sys_ref and RTL_B, provided their Σ labels, values, atomicity requirements, and required happens-before constraints are preserved.

The notation τ_ref,system ≈_sys τ_post,system is an abbreviation for CompatibleSys C τ_ref τ_post. It is a directed compatibility predicate, not a symmetric equivalence relation. The main construction direction is Post → Ref; the reverse direction is restricted to RTL-consistent source prefixes.

J.0:

```
theorem J0_pairwise_composition
  (C : ComparisonInstance)
  (W : WitnessBundle C)
  (hC : C.WellFormed)
  (hB : BRules C)
  (hR : RRules C)
  (hE : ERules C)
  (hIC : IssueCommitWF C W)
  (hContracts : ModelingContracts C)
  (τ_ref : Prefix C.Sys_ref)
  (τ_post : Prefix C.RTL_B)
  (hProc : ∀ P, CompatibleP C P τ_ref τ_post)
  (hChan : ChannelsWellFormedAndMatched C τ_ref τ_post) :
  CompatibleSys C τ_ref τ_post
```

J.1:

```
structure JAssumptions
```

(C : ComparisonInstance)
(W : WitnessBundle C) where
wellformed : C.WellFormed
bRules : BRules C
rRules : RRules C
eRules : ERules C
issueCommitWF : IssueCommitWF C W
implementation : ImplementationAssumptions C
witnessCompatible : WC C W
contracts : ModelingContracts C
environment : ReactiveEnvironmentWF C.env

theorem J1_post_to_ref
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JAssumptions C W)
(τ_post : Prefix C.RTL_B)
(hExec : Exec_post C τ_post) :
∃ τ_ref,
Exec_ref C τ_ref ∧
CompatibleSys C τ_ref τ_post

Restricted reverse:
theorem J1_ref_to_post_restricted
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JAssumptions C W)
(τ_ref : Prefix C.Sys_ref)
(hExec : Exec_ref C τ_ref)
(hConsistent : RTLConsistentSourcePrefix C W τ_ref) :
∃ τ_post,
Exec_post C τ_post ∧
CompatibleSys C τ_ref τ_post

Cutpoint correspondence:

```
structure CutpointCorrespondence
 (C : ComparisonInstance)
 (σ_ref : State)
 (σ_post : State) where

 nextfront_agreement : Prop
 frontier_guard_value_boundaryreq_agreement : Prop
 buffered_channel_state_agreement : Prop
 raw_rendezvous_endpoint_request_agreement : Prop
 pending_blocking_enablement_agreement : Prop

 direct_input_anchor_agreement : Prop
 array_mapping_agreement : Prop
 r2c_fixed_point_agreement : Prop

theorem J1_cutpoint_correspondence
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JAssumptions C W)
(τ_ref : Prefix C.Sys_ref)
(τ_post : Prefix C.RTL_B)
(hCompat : CompatibleSys C τ_ref τ_post)
(hNorm : EpsilonNormalizedPair C τ_ref τ_post) :
∃ σ_ref σ_post,
TerminalCutpoint τ_ref σ_ref ∧
TerminalCutpoint τ_post σ_post ∧
CutpointCorrespondence C σ_ref σ_post
```

---

18. Appendix K WFG and liveness

Define WFG over the selected proof-internal Sys_ref / RTL_B boundaries, not over the outer Sigma_DUT projection. In ordinary cases this distinction may collapse to the identity projection. In elaborated cases, however, Sys_ref = Sys_B^elab(E), and the WFG must see the explicit staged/bundle/join/epoch-gate abstract channels introduced by E even when sigmaDUTProjection / Π_elab hides their events from the outer DUT/testbench-visible alphabet.

```
def FrontierItemChannelSide
```

```
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem)
(c : Channel) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
q.channel = c

def ComplementOwnerItemSide
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem)
(Q : Process) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
ComplementOwner C side q = Q

def InternalChannelOfFrontierItem
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
InternalChannel C side q.channel

def WFGProofInternalBoundaryItem
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
q.channel ∈ ProofInternalChannels C side ∧
ChannelRepresentedInSigma C.Sigma q.channel ∧
¬ ChannelVisibilityDefinedOnlyBySigmaDUT C q.channel

def WFG
(C : ComparisonInstance)
(side : Side)
```

```
(σ : State) : Digraph Process :=
{ edge := WFGEdge C side σ }

def WFGEdge
(C : ComparisonInstance)
(side : Side)
(σ : State)
(P Q : Process) : Prop :=
∃ x,
x ∈ Front C side P σ ∧
BlockingMessageFrontierItem x ∧
BoundaryReqItemSide C side x σ ∧
ChannelDisabledItemSide C side x σ ∧
ComplementOwnerItemSide C side x Q ∧
InternalChannelOfFrontierItem C side x ∧
WFGProofInternalBoundaryItem C side x

theorem WFG_ignores_sigmaDUTProjection
(C : ComparisonInstance)
(side : Side)
(σ : State)
(P Q : Process) :
WFGEdge C side σ P Q ↔
WFGEdgeComputedOverProofInternalSigma C side σ P Q :=
by
-- WFG, Front, BoundaryReqItemSide, ChannelEnabledItemSide, and
-- ChannelDisabledItemSide are interpreted over C.Sigma and the chosen
-- Sys_ref / RTL_B abstract channels through the owning dynamic message
-- instance of each message-operation frontier item. sigmaDUTProjection is used
-- only after the proof-internal comparison when projecting to the outer
-- DUT/testbench-visible observation boundary.
sorry
```

Edges are generated from Front_P(σ), not arbitrary non-frontier asserted requests. This intentionally makes the WFG sparser than a graph over all asserted boundary requests: only pending blocking frontier items generate wait-for edges. For message-operation frontier items, BoundaryReqItemSide and ChannelDisabledItemSide recover the owning DynMsgInstance and then apply the

operation-attributive BoundaryReqSide / ChannelDisabledSide predicates. Failed non-blocking polls and already-asserted non-frontier requests may affect state or occupancy through other rules, but they do not themselves create WFG dependency edges.

The Lean development must include the Appendix K.2.2 / Appendix R.1 structural classification theorem as an explicit milestone. Front_P(σ) is not an independent primitive for liveness proofs: at ε-quiescent cutpoints it is derived from the blocking message-operation slice of NextFront_P together with the operation-attributive BoundaryReq predicate.

```
def BlockingMsgNextFront
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State) : Set FrontierItem :=
{ x |
x ∈ NextFront C side P σ ∧
x ∈ QOmegaOfProcess C side P σ ∧
BlockingMessageFrontierItem x ∧
MessageKindOfFrontierItem C side x ∈ {MsgKind.Push, MsgKind.Pop} }

def FrontFromNext
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State) : Set FrontierItem :=
{ x |
x ∈ BlockingMsgNextFront C side P σ ∧
BoundaryReqItemSide C side x σ }

theorem K0_front_nextfront_classification
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State)
(hK : DocKAssumptions C W)
(hQ : EpsQuiescent σ)
```

```
(hWF : WellFormedCutpoint C side σ) :
Front C side P σ =
FrontFromNext C side P σ :=
by
-- Appendix K.2.2 / Appendix R.1 proof target.
-- Uses Appendix B drain-only / no-withdrawal blocking discipline and
-- Appendix C Issue/Commit attribution to show that the asserted blocking
-- message-operation slice of NextFront_P is exactly the current pending
-- blocking frontier. BoundaryReqItemSide reads each message-operation
-- frontier item through its owning dynamic message instance.
sorry

theorem K0_front_correspondence_from_cutpoint
(C : ComparisonInstance)
(P : Process)
(σ_ref σ_post : State)
(hK : DocKAssumptions C W)
(hCorr : CutpointCorrespondence C σ_ref σ_post)
(hQref : EpsQuiescent σ_ref)
(hQpost : EpsQuiescent σ_post) :
Front C .ref P σ_ref = Front C .post P σ_post :=
by
-- Apply K0_front_nextfront_classification on both sides, then use the
-- CutpointCorrespondence clauses for NextFront agreement and
-- operation-attributive BoundaryReqItemSide agreement on the common
-- BlockingMsgNextFront slice.
sorry
```

This theorem is intentionally stated for BlockingMsgNextFront_P, not for the broader MsgNextFront_P. Successful non-blocking message-operation frontier items may contribute committed Push_c / Pop_c Σ-labels, and failed non-blocking polls are ε-steps, but neither creates a pending blocking WFG cause.

```
def ProcessStalledAtFront
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State) : Prop :=
```

```
Front C side P σ ≠ ∅ ∧
∀ x,
x ∈ Front C side P σ →
BlockingMessageFrontierItem x →
BoundaryReqItemSide C side x σ →
ChannelDisabledItemSide C side x σ

def WFGClosed
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
(σ : State) : Prop :=
∀ P Q,
P ∈ D →
WFGEdge C side σ P Q →
Q ∈ D

def WFGTerminalSCC
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
(σ : State) : Prop :=
D.Nonempty ∧
StronglyConnectedSubgraph (WFG C side σ) D ∧
WFGClosed C side D σ

def DrainClosed
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
(σ : State) : Prop :=
∀ P,
P ∈ D →
NoEnabledDrainStepLeavingD C side P D σ

def ExistsClosedDrainClosedWFGCycle
(C : ComparisonInstance)
(side : Side)
```

(σ : State) : Prop :=
∃ D,
WFGTerminalSCC C side D σ ∧
DrainClosed C side D σ ∧
∀ P,
P ∈ D →
ProcessStalledAtFront C side P σ

def InternalMPDeadlockAt
(C : ComparisonInstance)
(side : Side)
(σ : State) : Prop :=
EpsQuiescent σ ∧
ExistsClosedDrainClosedWFGCycle C side σ

The K.1 SCC contradiction should use this definition. The closed SCC supplies the cycle of internal message-passing dependencies; DrainClosed rules out escape by downstream draining; ProcessStalledAtFront ensures every process in D is stalled on its current blocking frontier, not merely carrying a non-frontier asserted request.

K assumptions should preserve the document's A1–A11 as a bundle, and add Lean-side auxiliary contracts without pretending they are doc-numbered assumptions.

def A11_Kepsilon_WFGInert
(C : ComparisonInstance) : Prop :=
∀ side P τ t_0 t $σ_0$ σt,
StateAtCycleInPrefix C side τ t_0 $σ_0$ →
StateAtCycleInPrefix C side τ t σt →
t_0 ≤ t →
InternalOrEpsilonOnlyBetween C side τ t_0 t →
FrontierBlocked C side P $σ_0$ →
FrontierRemainsBlockedOver C side P τ t_0 t →
WFGInertForProcess C side P $σ_0$ σt ∧
NoNewBlockingFrontierCause C side P $σ_0$ σt

structure DocKAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where

A1_point_to_point_channels : Prop
A2_chosen_Sys_ref_boundaries : Prop
A3_R_E_rule_conformance : RRules C ∧ ERules C
A4_B_rules : BRules C
A5_source_liveness : InternalMPDeadlockFree C.Sys_ref
A6_issue_commit_wf : IssueCommitWF C W
A7_witness_bundle : WC C W
A8_epsilon_normalization : Prop
A9_single_transfer_per_cycle : Prop
A10_reset_empty_or_initial_accounting : Prop
A11_Kepsilon_WFG_inert :
A11_Kepsilon_WFGInert C

def KElaborationPrerequisite
(C : ComparisonInstance) : Prop :=
∀ c,
ChannelCanContributeWFGEdge C c →
ChosenAbstractBoundaryOfSysRef C c ∧
NoHiddenSyncBoundaryGatingAtOrdinaryBufferedBoundary C c ∧
(RequiresSyncBoundaryWithholding C c →
C.refChoice.kind = ReferenceKind.elaborated)

Lean-side additions:
structure KAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where
doc : DocKAssumptions C W
contracts : ModelingContracts C
r2c? : Option (R2C_WF C)
elaboration_prerequisite :
KElaborationPrerequisite C

This avoids fictional A12/A13 labels.

K lemmas:
theorem K0_frontier_blocking_classification
(C : ComparisonInstance)

(W : WitnessBundle C)
(hK : KAssumptions C W)
(σ : State)
(hQ : EpsQuiescent σ) :
...

theorem K0a_rendezvous_endpoint_request_agreement
(C : ComparisonInstance)
(W : WitnessBundle C)
(hCorr : CutpointCorrespondence C σ_ref σ_post)
(c : Channel)
(hB0 : capacity c = 0) :
Req_prod c σ_ref = Req_prod c σ_post ∧
Req_cons c σ_ref = Req_cons c σ_post

theorem K1_deadlock_characterization
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W)
(σ : State)
(hReach : Reachable C.RTL_B σ)
(hQ : EpsQuiescent σ) :
InternalMPDeadlockAt C .post σ ↔
ExistsClosedDrainClosedWFGCycle C .post σ

This theorem uses hK.elaboration_prerequisite: every channel boundary used by the WFG is the chosen abstract boundary of the Sys_ref / RTL_B comparison after any required elaboration. Synchronization-boundary ramp-down / epoch-gating is therefore represented at explicit Sys_B^elab channels, not as a hidden exception to E4 on an otherwise ordinary buffered channel.

theorem K2_dependency_refinement
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W)
(σ_ref σ_post : State)
(hCorr : CutpointCorrespondence C σ_ref σ_post) :

WFG C .ref σ_ref = WFG C .post σ_post

theorem K2a_drain_closure_transfer
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W)
(D : Finset Process)
(σ_ref σ_post : State)
(hCorr : CutpointCorrespondence C σ_ref σ_post)
(hDrainPost : DrainClosed C .post D σ_post) :
DrainClosed C .ref D σ_ref

Exec-post membership at a deadlock cutpoint should be explicit:

theorem deadlock_cutpoint_in_Exec_post
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W)
(σ_post : State)
(hReach : Reachable C.RTL_B σ_post)
(hDead : ExistsClosedDrainClosedWFGCycle C .post σ_post)
(hIdle : C.assumptions.B.B6_idleStutterTimeDivergence) :
∃ τ_post,
TerminalCutpoint τ_post σ_post ∧
Exec_post C τ_post

Main K theorem:
theorem K1_liveness_preservation
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W) :
InternalMPDeadlockFree C.Sys_ref →
InternalMPDeadlockFree C.RTL_B

This is intentionally one-way. Do not state:

InternalMPDeadlockFree C.Sys_ref ↔ InternalMPDeadlockFree C.RTL_B

---

19. Appendix R facade

Appendix R should be formalized last as aliases/wrappers over earlier theorems.

Use Lean names that avoid the doc's R.2 collision:

theorem R_lemma1_front_nextfront_classification := ...
theorem R_theorem1_prefix_matching := ...
theorem R_corollary1_cutpoint_correspondence := ...
theorem R_corollary1a_rendezvous_endpoint_request_agreement := ...
theorem R_lemma2_dependency_refinement := K2_dependency_refinement
theorem R_theorem2_closed_cycle_and_one_way_drain_transfer := ...

Do not call cutpoint correspondence R1_cutpoint_correspondence; in the doc structure, it corresponds to a corollary, not Theorem R.1.

---

20. What remains as hypotheses

These remain theorem parameters or design/tool obligations:

1. R-rules R1, R2, R2C, R3, R4, R5.
2. E-rules E1–E5.
3. B1–B6, with B2/B3 encoded temporally.
4. Implementation assumptions such as I.A0.
5. Witness compatibility WC, derived from Lemma L.2 or Corollary L.3 where applicable.
6. Direct-input contracts.
7. Array access mapping rules.
8. Elaboration package E, when Sys_ref = Sys_B^elab(E).
9. Source-level internal message-passing deadlock freedom for K.

---

21. What should be proved inside Lean

Lean should prove:

- bucketed execution infrastructure;
- temporal lemmas for AlwaysFrom / EventuallyFrom;

- channel-capacity lemmas for rendezvous and buffered cases;
- issue existence/uniqueness and derived IssueTime;
- direct-input late-sampling preservation from primitive environment contracts;
- array sync-boundary commit-order preservation;
- R2C fixed-point observation lemmas from R2C_WF;
- automatic-flush frontier lemmas;
- Lemma L.2: snooping establishes WC;
- Corollary L.3: NB-CF identity witness;
- I.DR over concrete ObsSigma;
- Appendix I finite and infinite witness construction;
- Proposition J.0;
- Theorem J.1 post-to-reference;
- restricted reverse theorem for RTL-consistent source prefixes;
- Corollary J.1 cutpoint correspondence;
- K.0/K.0a/K.1/K.2/K.2a;
- deadlock cutpoint membership in Exec_post;
- Theorem K.1 one-way liveness preservation;
- Appendix R aliases.

---

22. Development milestones

Milestone A — Sequential, blocking-only, non-elaborated core

Scope:

- Sys_ref = Sys_B;
- sequential processes only;
- no NB calls;
- no snooping;

- no external arrays;
- no direct-input late sampling;
- no combinational R2C;
- bucketed execution model.

Targets:

J1_post_to_ref_blocking
J1_cutpoint_correspondence_blocking
K1_liveness_preservation_blocking

Milestone B — Temporal B2/B3

Add:

- infinite bucketed executions;
- AlwaysFrom, EventuallyFrom;
- concrete FairAction;
- P_push, P_pop.

Target:

K1_liveness_preservation_with_temporal_progress

Milestone C — Non-blocking under NB-CF

Add:

- Qexec;
- failed NB polls as ε;
- successful NB transfers as Σ;
- Corollary L.3.

Target:

J1_post_to_ref_nb_cf

Milestone D — Witness selector / snooping

Add:

- partial-prefix WitnessSelector;

- Lemma L.2.

Target:

J1_post_to_ref_with_snooping

Milestone E — Direct inputs

Add:

- DirectInputContract;
- derived lateSamplingPreservesLogicalAnchor;
- bucket placement for logical anchor vs physical late sample.

Target:

J1_post_to_ref_direct_inputs

Milestone F — External arrays

Add:

- MemoryEvent;
- ArrayAccessMapping;
- sync-relative array commit rules.

Target:

J1_post_to_ref_external_arrays

Milestone G — R2C

Add:
· combModel?;
· LocalCombNetwork;
· CombNetwork;
· CombComposition;
· R2C_WF;
· per-process combinational component obligations;
· composed fixed-point bucket observation semantics.
Target:
J1_post_to_ref_with_R2C

Milestone H — Elaboration

Add:

- ElaborationPackage Sys_B Chan_outer Σ_DUT;
- Π_elab;
- π_Σ,E;
- A_E;
- explicit-channel interpretation of B, occ, BoundaryReq, ChannelEnabled, Front, and WFG.

Targets:

J1_post_to_ref_elab
K1_liveness_preservation_elab

Milestone I — Appendix R facade

Add compact R theorem wrappers after the full proof stack is available.

---

23. Expected difficulty

The hardest pieces remain:

1. Bucketed execution and ideal induction.
2. Temporal B2/B3 over infinite executions.
3. Concrete ObsSigma and I.DR.
4. Appendix J finite-ideal construction.
5. Direct-input late-sampling contracts.
6. External array access mapping.
7. R2C fixed-point semantics.
8. Appendix Q elaboration obligations.
9. Appendix K Exec_post membership at deadlock cutpoints.

A polished full formalization is plausibly in the 18k–30k Lean line range if direct inputs, arrays, R2C, temporal fairness/progress, non-blocking witness selection, snooping, automatic flush, Issue/Commit attribution, A11/Kε, and

elaboration are all included. A core sequential/blocking/non-elaborated proof would be substantially smaller, plausibly 6k–10k lines; an intermediate blocking-plus-temporal-WFG proof would likely fall around 10k–16k lines.